DESIGN AND IMPLEMENTATION OF AN sEMG-BASED GESTURE-CONTROLLED ROBOTIC ARM USING DEEP LEARNING



By

SALMA FARID

A Thesis Submitted to the Faculty of Engineering,

The British University in Egypt, (**BUE**)

In Partial Fulfilment of the Requirements for

The **Bachelor of Engineering (BEng)** Degree in

Robotics Engineering

June 2026

DESIGN AND IMPLEMENTATION OF AN sEMG-BASED GESTURE-CONTROLLED ROBOTIC ARM USING DEEP LEARNING

Thesis Approved

Thesis Advisor Name	**Signature / Date**
----------	**----------**

Internal Committee Member	**Signature / Date**
----------	**----------**

External Committee member	**Signature / Date**
----------	**----------**

Head of Mechanical Engineering Department

Faculty of Engineering. The British University in Egypt

Name _____

Signature _____ **Date** _____

Declaration of Authorship

I hereby declare that this work titled “Design and implementation of an sEMG-based gesture-controlled robotic arm using deep learning” submitted to the Mechatronics Engineering department of the British University in Egypt is an original piece of work and has not been published or showcased anywhere else in the past. This work was carried out under the supervision of ….

Name

Signature / Date _____

Acknowledgements

I would like to express my deep gratitude to Dr.Nourhan Zayed and Dr. Mostafa Abdelaziz, my graduation research supervisors and my mentors, for their patient guidance, enthusiastic encouragement and useful critiques of this research work

I would also like to thank Eng. Mai Akram for her support while carrying out the lab work, my colleagues Maya and Mahmoud for working alongside me for the better part of a year and being there to brainstorm ideas and solve problems encountered during this project especially during the lab work.

I am also very grateful to the British University of Egypt for providing the opportunity to carry out this research and the resources necessary to carry it out successfully.

Finally, I would like to thank my parents for always supporting and encouraging my education. Their hard work and patience is the reason I am where I am today.

Abstract

Robots that work alongside people need to read human intent, and they need to do it quickly. Surface electromyography (sEMG) is one way to capture that intent without anything invasive, since it records the electrical activity of contracting muscle. The difficulty is that sEMG is noisy and changes from moment to moment, which makes some gestures hard to tell apart, especially low-effort ones like rest, grip, and finger adduction that look alike electrically. This thesis covers the design and build of an sEMG-driven robotic manipulator that uses deep learning for recognition, with ROS2 handling the control layer and MoveIt2 the kinematics. Features were taken from a mix of Mel-Frequency Cepstral Coefficients and time-domain descriptors so that the temporal shape of each muscle activation is captured. A multi-scale CNN-BiLSTM with channel attention then learns how those features evolve in space and time. The experiments used ten hand and wrist gestures recorded from 40 participants across four sEMG channels at 2000 Hz, with data augmentation and post-processing logic added to cope with differences between people and with rest-state errors.

Keywords sEMG, gesture recognition, CNN-BiLSTM, channel attention, ROS2, MoveIt2, robotic manipulator, deep learning

Table of Contents

Acknowledgements iv

Abstract v

List of Figures ix

List of Tables x

Nomenclature xi

Abbreviations xii

Chapter (1) Introduction 1

1.1 Background 1

1.2 Research Hypothesis 1

1.3 Aim and Objectives 2

1.4 Thesis Outline 2

Chapter 1 3

Chapter (2) Literature Review 3

2.1 Introduction 3

2.2 sEMG gesture detection research 3

2.2.1 Overview of sEMG-Based Human-Robot Interfaces 3

2.2.2 Deep Learning Paradigms for sEMG Gesture Recognition 4

2.2.3 Multimodal and Adaptive Systems 4

2.2.4 The Challenge of the Rest State 4

2.2.5 sEMG in Robotic Manipulation and ROS2 Integration 5

2.2.6 Design Space and Physiological Constraints 5

2.2.7 Comparative Summary of Key Studies 5

2.3 Conclusion 9

Chapter (3) Research Strategy 10

3.1 Data 10

3.2 Research Methods 10
3.3 Preprocessing 12
3.3.1 Model Architectures and Hyperparameter Configuration 13
3.3.2 Deep Learning Training Configuration 14
3.4 Overview 15
3.4.1 Multi-scale CNN-BiLSTM with channel attention 15
3.4.2 Triple-scale CNN + dual BiLSTM with STAE gating 18
3.4.3 STFT spectrogram with dual path 2D CNN 21
3.4.4 Random Forest 22
3.4.5 Support Vector Machine (SVM) 23
3.4.6 Gradient Boosting 24
3.4.7 Recurrent CNN (CRNN) 25
3.5 Post-Processing and resultant evaluation 25
Chapter (4) Experimental Work 28
4.1 Mechanical Design 28
4.1.1 Introduction 28
4.1.2 Conceptual Design 29
4.1.3 Detailed Design 29
4.1.4 Discussion and Conclusion 32
4.2 Electrical and Electronics Design 32
4.2.1 Introduction 33
4.2.2 Design Methodology 33
4.2.3 Detailed Design 34
4.2.4 Discussion and Conclusion 36
4.3 Control Design and Implementation 36
4.3.1 Simulation and Validation 41
4.3.2 Conclusion and Future Work 42

4.4 EMG Acquisition 44
4.4.1 Collection Software and Recording Protocol 44
4.4.2 Electrode Placement 46
4.4.3 Data-Collection Campaign and Data Validity 47
4.4.4 Analysis and Simulation Pipeline 47
4.4.5 Feature Extraction and Model Pipeline 48
4.4.6 End-to-End Test Configuration 50
4.4.7 Mains-Interference Experiment 51
4.4.8 Evaluation Methodology and a Corrected Leakage Pitfall 51
4.4.9 Gesture-Subset Selection 51
4.5 Control System 52
4.5.1 Introduction 52
4.5.2 System Modelling and Analysis 52
4.6 Prototyping and Testing 55
4.6.1 Motion-Control Testing 55
4.7 Results 55
4.7.1 Classification Accuracy 55
4.7.2 Per-Class Performance of the Deployed Model 56
4.7.3 Feature Separability 58
4.7.4 Simulation and System Validation 59
4.7.5 Summary of Findings 60
Chapter (5) Analysis and Discussion 61
5.1 Summary of Findings 61
5.2 Interpretation of Results 61
5.3 Discussion 62
5.1 5.4 Limitations of Study 63
Chapter (6) Conclusions 65

6.1 Conclusions 65
6.2 Future Work 66
Appendix A – Hardware specifications 67

List of Figures

Figure 1 - System Methodology flowchart 11
Figure 2 - Lab acquired normalized signals 12
Figure 3 - Proposed Multi-scale CNN-BiLSTM Architecture 16
Figure 4 - CNN-BiLSTM Confusion Matrix 17
Figure 5 CNN-BiLSTM per-class performance 18
Figure 6 CNN-BiLSTM confusion matrix for lab acquired data 19
Figure 7 - STFT 2D-CNN confusion matrix for lab acquired data 21
Figure 8 - RF Confusion matrix for lab acquired data 22
Figure 9 - SVM Confusion matrix for lab acquired data 23
Figure 10 - GBM confusion matrix for lab acquired data 24
Figure 11 - CRNN confusion matrix for lab acquired data 25
Figure 12 Dimensioned engineering drawing 30
Figure 13 PCA9865 Wiring diagram 35
Figure 14 - Control architecture block diagram 37
Figure 15 sEMG data flow 40
Figure 16 Data-collection application showing the gesture cue 45
Figure 17 - Raw gesture signals 46
Figure 18 sEMG sensor placement on the forearm flexor and extensor muscle groups 47
Figure 19 The fabricated 6-DOF arm and gripper 50
Figure 20 Model comparison 56
Figure 21 Confusion matrix of the deployed random forest on the held-out test cycle 58
Figure 22 Top features ranked by ANOVA F-value (class separability) 59

List of Tables

Table 1- Summary of Representative sEMG Gesture Recognition Studies 6
Table 2 Summary of Control Schemes implemented in studies 8
Table 3 Extracted features for machine learning algorithms 13
Table 4 - Model Architectures and Key Hyperparameters 14
Table 5 - Deep Learning Training Hyperparameters 14
Table 6 Highly confused pairs 19
Table 7 Joint Specifications of the 6-DOF Robotic Arm 29
Table 8 Link Mass Properties 31
Table 9 Muscle Sensor v3 Specifications 34
Table 10 - Hardware Control Software Modules 38
Table 11 Teleoperation Keybindings 39
Table 12 - Feature extraction 48
Table 13 Actuated Joints of the 6-DOF Arm 53
Table 14 Servo control specifications 53

Nomenclature

c	Servo channel index
k	Angular gain (servo degrees per radian of joint motion)
R	Gain-setting resistor of the Muscle Sensor v3
V_{in}	Input voltage to the resistive divider (sensor SIG output)
V_{out}	Output voltage of the resistive divider (ESP32-S3 ADC input)
V_{s}	Sensor supply voltage (±9 V)
θ	Commanded joint angle (radians)

Abbreviations

ABS	Acrylonitrile Butadiene Styrene
ADC	Analogue-to-Digital Converter
API	Application Programming Interface
BiLSTM	Bidirectional Long Short-Term Memory
CA	Channel Attention
CAD	Computer-Aided Design
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
DC	Direct Current
DDS	Data Distribution Service
DOF	Degree of Freedom
EMG	Electromyography
FDM	Fused Deposition Modelling
GPIO	General-Purpose Input/Output
I²C	Inter-Integrated Circuit
IC	Integrated Circuit
IK	Inverse Kinematics
IMU	Inertial Measurement Unit
JTC	Joint Trajectory Controller
KDL	Kinematics and Dynamics Library
LDA	Linear Discriminant Analysis

LSTM	Long Short-Term Memory
LTS	Long-Term Support
mDNS	Multicast Domain Name System
MFCC	Mel-Frequency Cepstral Coefficients
ML	Machine Learning
NMES	Neuromuscular Electrical Stimulation
PLA	Polylactic Acid
PSRAM	Pseudo-Static Random-Access Memory
PWM	Pulse-Width Modulation
QoS	Quality of Service
RMS	Root Mean Square
RNN	Recurrent Neural Network
ROS	Robot Operating System
RViz	ROS Visualization
sEMG	Surface Electromyography
SENIAM	Surface Electromyography for the Non-Invasive Assessment of Muscles
SPI	Serial Peripheral Interface
SVM	Support Vector Machine
TCN	Temporal Convolutional Network
TCP	Transmission Control Protocol
TDD	Time-Domain Descriptors
TRM	Technical Reference Manual
URDF	Unified Robot Description Format

Chapter (1) Introduction

1.1 Background

Human-robot interaction has spread into industrial and assistive settings, and that shift has moved robots away from fixed, pre-programmed routines toward something more responsive to the person using them. The hard part is the interface how do you turn what a person intends into actuation, especially on systems that demand precision? Joysticks and keyboards put a constant mental load on the operator, and that load only grows as the number of degrees of freedom rises. Surface electromyography offers a way around this, reading neuromuscular activity from the skin without anything invasive.

The catch is the signal itself. sEMG is stochastic and non-stationary, and neighbouring muscle groups bleed into each other through crosstalk, all of which makes a clean control signal hard to recover [1]. The preliminary results show one sharp example. Before any post-processing, the raw network output classified the rest state with only 50.77% accuracy, because rest looks much like other low-activation states such as gripping. In practice that means phantom movements, where the arm reads low-level muscle noise as a command.

There is also the software side. Feeding a biological signal into ROS2 means the Python deep-learning model and the C++ MoveIt2 planner have to stay tightly in step, or real-time response and arm stability both suffer. The work in this thesis pulls these pieces into one architecture that covers acquisition, classification, and motion planning on a custom 6-DOF manipulator.

1.2 Research Hypothesis

The main objective is a real-time control system that maps at least ten distinct hand gestures onto a robotic manipulator. Three research questions frame the investigation

1. Can a multi-scale CNN-BiLSTM architecture with channel attention achieve an overall offline validation accuracy exceeding 70% on a ten-class sEMG gesture dataset?
2. Can a hybrid MFCC and time-domain feature extraction pipeline provide sufficient inter-class discriminability to separate physiologically similar gestures, such as rest, grip, and adduction?
3. Can the classified gesture output be reliably mapped to 6-DOF joint trajectories through a ROS2 and MoveIt2 control architecture with latency within perceptual limits?

The null hypothesis holds that classification accuracy does not differ significantly between the proposed multi-scale CNN-BiLSTM and a baseline single-scale CNN trained on the same data. Rejecting the null would support the value of multi-scale feature extraction and bidirectional temporal modelling for this recognition problem.

1.3 Aim and Objectives

The aim is to take the principles of multi-channel surface electromyography and turn them into a working methodology locate muscle innervation zones, fix a reliable electrode-placement protocol, and build a control bridge accurate enough for a person to drive a robotic arm intuitively, in real time, with low latency and good separation between gestures.

The technical objectives are as follows

1. Develop a robust deep learning classification model that achieves an overall accuracy exceeding 80% on a ten-gesture sEMG dataset collected from 30 or more participants.
2. Integrate the classification output with a ROS2-based trajectory controller for the 6-DOF robotic arm using the MoveGroup interface in MoveIt2.
3. Simulate gesture-driven trajectories in Gazebo using the URDF model derived from the SolidWorks CAD assembly to validate motion planning algorithms prior to hardware deployment.
4. Optimise motion planning and state-machine logic to maintain stability between gesture transitions and prevent phantom actuation from misclassified rest states.
5. Implement and validate a hardware prototype to demonstrate intuitive sEMG-driven control of the physical 6-DOF robotic arm.

1.4 Thesis Outline

The rest of this thesis is laid out as follows. Chapter 2 reviews the literature on sEMG-based gesture recognition, deep-learning architectures, multimodal control, and ROS2 robotic integration. Chapter 3 sets out the research strategy the data, the design of the mechanical, electrical, and control subsystems, and the plan that ties them together. Chapter 4 covers the experimental work, from the data-collection protocol and signal preprocessing through to the model and its results. Chapter 5 analyses and discusses the classification and integration findings, and Chapter 6 draws conclusions and sets out directions for future work.

Chapter (2) Literature Review

This chapter reviews the literature behind an sEMG-based gesture-controlled robotic manipulator. It is organised around five themes sEMG-based human-robot interfaces, deep-learning approaches to gesture recognition, multimodal and adaptive systems, the recurring problem of the rest state, and the integration of sEMG with robotic manipulation frameworks. It closes with a comparative summary of the main studies, and the research gaps this project sets out to fill.

2.1 Introduction

This section sets the scope of the review and places the work within what is already known about sEMG-based human-robot control, deep-learning gesture recognition, and ROS2 integration. The studies covered run from 2005 to 2025, with weight given to those that bear directly on a ten-gesture classification task and 6-DOF manipulation. The scope stays with wired and wireless sEMG systems aimed at forearm and wrist gestures; exoskeleton and prosthetic systems appear only where their methods carry over.

2.2 sEMG gesture detection research

The review is organised by theme. It moves from sEMG-based human-robot interfaces and the deep-learning methods used to decode them, through multimodal and adaptive systems, and on to two issues that bear directly on this project the rest state and the integration of gesture recognition with a robot running under ROS2.

2.2.1 Overview of sEMG-Based Human-Robot Interfaces

Surface electromyography has long been a key sensor for reading movement intent in assistive and industrial robotics. Where a joystick or keyboard is an indirect controller, sEMG ties muscular activation straight to robotic actuation [1]. Its signals are nonlinear, stochastic, and full of crosstalk between muscles, which makes real-time classification hard [2].

Early systems leaned on time- and frequency-domain descriptors fed to classical classifiers such as linear discriminant analysis (LDA) and support vector machines (SVM). These worked for small gesture sets but did not scale well to multi-DOF manipulators. Deep learning moved the field toward data-driven feature learning, which generalised more reliably across users and electrode layouts.

2.2.2 Deep Learning Paradigms for sEMG Gesture Recognition

Convolutional networks have proven good at pulling spatial patterns out of multichannel sEMG. [3] reported 99.99% accuracy on CapgMyo with a lightweight CNN using just two hidden layers, which shows that efficiency and accuracy can go together. [4] combined a multi-stream CNN with time-varying temporal convolutional networks (TCN) for 98.96% on Ninapro DB9; channel attention let the network lean on physiologically relevant electrodes and damp noise from inactive muscles.

Temporal modelling matters when gestures share spatial features. [5] built a CNN-RNN hybrid that recognised 20 gestures in real time at an average 153 ms latency, within the limits of human perception. [6]showed CNN-LSTM pipelines beating CNN-only networks by more than 10% in accuracy, and with post-processing a gesture could be called from only 200 ms of EMG, which opens the door to anticipatory robot responses. BiLSTM layers, as set out in Hassan Abdelmoneam's research, read temporal dependencies in both directions, which helps pin down when a contraction starts and ends [7].

[8]used channel attention to weight individual electrodes on the fly, quieting noisy channels and favouring informative ones, which helped with inter-channel variability and electrode misalignment. Transformers have been tried too [9] reached 82.93% on Ninapro DB2 with TEMGNet and self-attention, and [10] hit 98.65% on CapgMyo with an LSTM-EMG-Net that fuses short- and long-term features through cross-attention.

2.2.3 Multimodal and Adaptive Systems

Relying on sEMG alone leaves a system exposed to electrode shift, muscle fatigue, and sweat artefacts [11]. [12] built a CG-Recogniser that combined sEMG with IMU sensors and a YOLOv5 camera stage, holding above 98% accuracy for continuous recognition. Sul et al. (2025) found that multimodal fusion lifted accuracy from 81.6% to 95.3% over an sEMG-only model. [13]ran a hybrid EMG-NMES system that cut muscle fatigue by 28.6% and steadied grip. Together these point to adaptive feedback and multimodal input as important for manipulation tasks that must last.

2.2.4 The Challenge of the Rest State

The rest state is one of the harder classification problems, since separating it from low-intensity gestures has proven consistently awkward. [1] found baseline noise and weak contractions sharing spectral space, a result backed by[14], who saw systematic rest-grip confusion in high-density datasets. [15] added a locking-gesture mechanism that pushed accuracy from 90.6% to

98%. [6] showed that post-processing, such as temporal buffering and softmax confidence thresholding, cut false activations sharply. These ideas feed directly into the post-processing logic used here.

2.2.5 sEMG in Robotic Manipulation and ROS2 Integration

Early EMG-driven robotics centred on teleoperation. [16] drove a 7-DOF Mitsubishi PA-10 from EMG and position tracking, with a fatigue-resistant model that turned biceps sEMG into elbow torque [17] showed shared autonomy in assistive manipulation, reaching a 100% task success rate with a body-machine interface on a Kinova MICO arm. Fabregó (2025) controlled a UR5e with threshold-based RMS logic inside ROS and Gazebo. [18] pointed to ROS2 as deterministic middleware, using DDS for real-time communication and ros2_control to drive actuators through a tightly timed read-update-write loop. [19] recommended state-machine control for turning discrete gestures into continuous trajectories, which steadies the system against rapid switching.

The move from ROS1 to ROS2 Jazzy matters here because of the Data Distribution Service (DDS) at its core. In this build, the sEMG inference node sends gesture commands with a Best Effort QoS to keep latency low, while the MoveIt2 trajectory node uses a Reliable QoS, so no joint command is dropped mid-execution. That predictability is what ros2_control needs, running its 100 Hz loop to keep the joint states updated.

2.2.6 Design Space and Physiological Constraints

[NO_PRINTED_FORM] [20]Electrode placement has a direct effect on accuracy. [21] showed that sitting an electrode over an innervation zone causes signal cancellation and argued for placing it between the innervation zone and the tendon insertion. J. M. Dick et al. (2024) warned against reading EMG amplitude as force without a biomechanical model. Both reinforce the design constraints used here standardised electrode placement, joint limits, velocity scaling, and physiological feasibility.

2.2.7 Comparative Summary of Key Studies

Table 2.1 gathers the representative sEMG gesture-recognition studies from this chapter, comparing architecture, dataset, accuracy, and main contribution. Table 2.2 does the same for the control schemes used in robotic manipulation work. Taken together, the reviewed work supports the choices made here a multi-scale CNN-BiLSTM with channel attention for classification, hybrid MFCC and TDD features, post-processing confidence thresholding to resolve the rest state, and ROS2 with MoveIt2 as the control middleware.

Table 1- Summary of Representative sEMG Gesture Recognition Studies

Author & Year	*Architecture / Model*	*Dataset*	*Accuracy*	*Key Feature / Contribution*
Kishore et al. (2025) [8]	Hybrid CNN-BiLSTM	NinaPro DB2	96.45%	Channel Attention for dynamic feature prioritization.
Soosaimariyan et al. (2025) [22]	Generalized Linear DIM (GLDIM)	DualMyo / NinaPro DB5	99.27%	Uses v-ATM filtering and MFCC frequency-domain features.
Ari (2023)	CNN+LSTM+TDD	UCI (Hand Grip)	98.34%	Fuses Time Domain Descriptors with deep learning models.
Lv et al. (2021) [23]	SOM-RBF Network	MYO Armband	96.87%	Self-Organizing Mapping to find optimal hidden node centers.
Zheng et al. (2022) [12]	YOLOv5 (ST-CNN)	Custom (Continuous)	over 98%	Standard deviation-based segmentation for continuous gestures.
Rezaee et al. (2024) [24]	U-Net + MobileNetV2	Six standard databases	90.23% (Avg)	Bayesian optimization for tuning BiLSTM hyperparameters.

Barona López et al. (2024) *[6]*	CNN-LSTM	EMG-EPN-612	90.55%	Post-processing algorithms to filter prediction sequences.
Abdelmoneam & Zayed (2025) *[7]*	Multi-Scale CNN-BiLSTM	Custom (4-channel)	87%	Progressive receptive field hierarchy for spatiotemporal learning.
Ben Abdallah et al. (2025) *[13]*	SVM / Fuzzy Hybrid	Custom (Bionic Hand)	95.4%	Integrates NMES to achieve a 28.6% reduction in fatigue.
Wang et al. (2022) *[25]*	Dual-Channel CNN	Jiayang Coal Mine	91.7% (True Event)	Combines time-domain data with WPD coefficients.
Zhang et al. (2020) *[10]*	GRU-RNN	Myo (21 gestures)	89.6%	Achieves prediction with only 200 ms of data.
Pérez-Reynoso et al. (2022) *[2]*	Multilayer Perceptron (MLP)	Custom (1-channel)	High Sensitivity	Activate State Machines for 3-DOF robot position control.
Farrell & Weir (2008)	LDA (Feature-based)	Targeted/Untargeted	95%-97%	Proved that symmetric electrode arrays match targeted placement.

Table 2 Summary of Control Schemes implemented in studies

Author (Year)	*Robotic Platform & Degrees of Freedom (DoF)*	*Control Strategy & Middleware*	*Key Contributions & Constraints*
Artemiadis & Kyriakopoulos (2005/2010) [16], [26]	Mitsubishi PA-10 (7-DoF)	Hybrid EMG & Position Tracking	Developed a fatigue-resistant model that effectively converts biceps sEMG signals into elbow joint torque and angles, ensuring accuracy across different levels of contraction.
Jain et al. (2015) [27]	Kinova MICO Arm	Shared Autonomy & BMI	Achieved a 100% success rate in "picking and pouring" tasks by splitting control between high-level human intent and low-level robotic execution.
Fabregó (2025) [17]	Universal Robots UR5e (6-DoF)	Threshold-Based Control (RMS) in ROS/Gazebo	Implemented 3D control using a Python script that triggered movements once signals surpassed specific RMS thresholds.
Macenski et al. / General ROS2 (2022) [18]	Various High-DoF Platforms (e.g., Franka FR3, UR5e)	ROS2 Deterministic Middleware via DDS	Identified ROS2 as real-time, deterministic middleware. Utilizes the ros2_control framework to manage actuators through a strictly timed Read-Update-Write loop.

2.3 Conclusion

The review points clearly toward hybrid CNN-BiLSTM architectures with channel attention as the leading approach for accurate multi-class sEMG recognition. Deep learning beats classical methods consistently, most of all on gestures that share muscle synergies. Post-processing built on temporal buffering and confidence thresholding comes up repeatedly as the way to handle rest-state errors in real deployments. And ROS2 with MoveIt2 stands out as the right framework for deterministic, real-time gesture-to-trajectory mapping.

Three gaps stand out. First, little published work connects sEMG with a custom, fully 3D-printed 6-DOF manipulator. Second, few studies report end-to-end latency from signal acquisition all the way to physical actuation. Third, the problem of synchronising mechanically coupled but dynamically mismatched servos on low-cost arms is barely explored. This project takes on all three.

Chapter (3) Research Strategy

3.1 Data

This project is built on surface electromyography (sEMG) recorded from the forearm. Two channels were targeted, one over the extensor digitorum and one over the flexor carpi radialis, and the signals were digitised by an ESP32 analog front end. The label set covers ten wrist and hand gestures, which were later narrowed to a seven-class set for driving the arm. Every gesture was repeated over five cycles in each session so the same movements could be compared across time. The signal itself, the electrode arrangement and the recording campaign are described in detail in Chapter 4.

3.2 Research Methods

The method rests on three engineering subsystems that together turn a muscle signal into arm motion the mechanical structure of the manipulator, the electrical and electronics front end that captures and digitises the sEMG, and the control system that drives the joints. Each is described in turn in later chapters, followed by the signal-processing and learning used for gesture recognition. To optimize the machine learning used, multiple machine learning algorithms were tested and ran comparatively using lab acquired pre-built datasets. An experimental engineering methodology was implemented so that it brings machine learning, signal processing, and robotic motion planning under one architecture, organised in four phases data acquisition and preprocessing, feature learning and model training, robotic system integration, and performance evaluation. The design leans on established biomedical signal-processing practice [1] [2], and a preliminary machine-learning model was implemented first to validate the approach before real-time control of the 6-DOF arm was attempted.

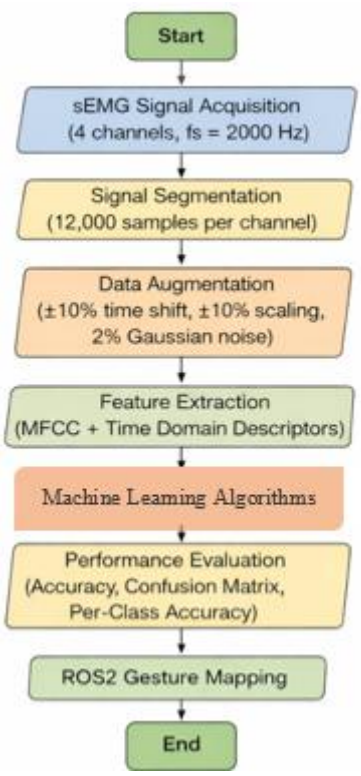


Figure 1 - System Methodology flowchart

Following Phinyomark & Scheme's 2018 study and Pérez-Reynoso's research in 2022, a supervised learning algorithm maps sEMG patterns to discrete gesture classes, and those classes are then translated into joint commands through ROS2 and MoveIt2.

Data acquisition came first. An initial sEMG dataset was recorded with sensors placed over the main extensor muscle groups that drive hand and wrist motion. Electrode positions followed SENIAM and innervation-zone-avoidance guidelines to keep phase cancellation and crosstalk down, drawing on the placement work of [21]. Signals were sampled at a fixed 2000 Hz and passed to an ESP32 for digitisation and transmission to the host computer.

Ten hand and wrist gestures were recorded rest, grip, flexion, extension, radial and ulnar deviation, supination, pronation, abduction, and finger adduction. Each was captured under controlled conditions, with visual cues guiding the participant. The initial set had 40 participants and six repetitions per movement, giving 135 samples per class, which kept the dataset balanced and limited class bias. Every sample was fixed at 12,000 time points, so each input to the network had the same sequence length. Rest was interleaved between gestures to capture baseline physiological noise and to limit fatigue, an approach shown to work well in recent sEMG studies [28]. Signals were then cut into overlapping time windows to trade

temporal resolution against latency [10], and z-score normalisation reduced amplitude drift between sessions.

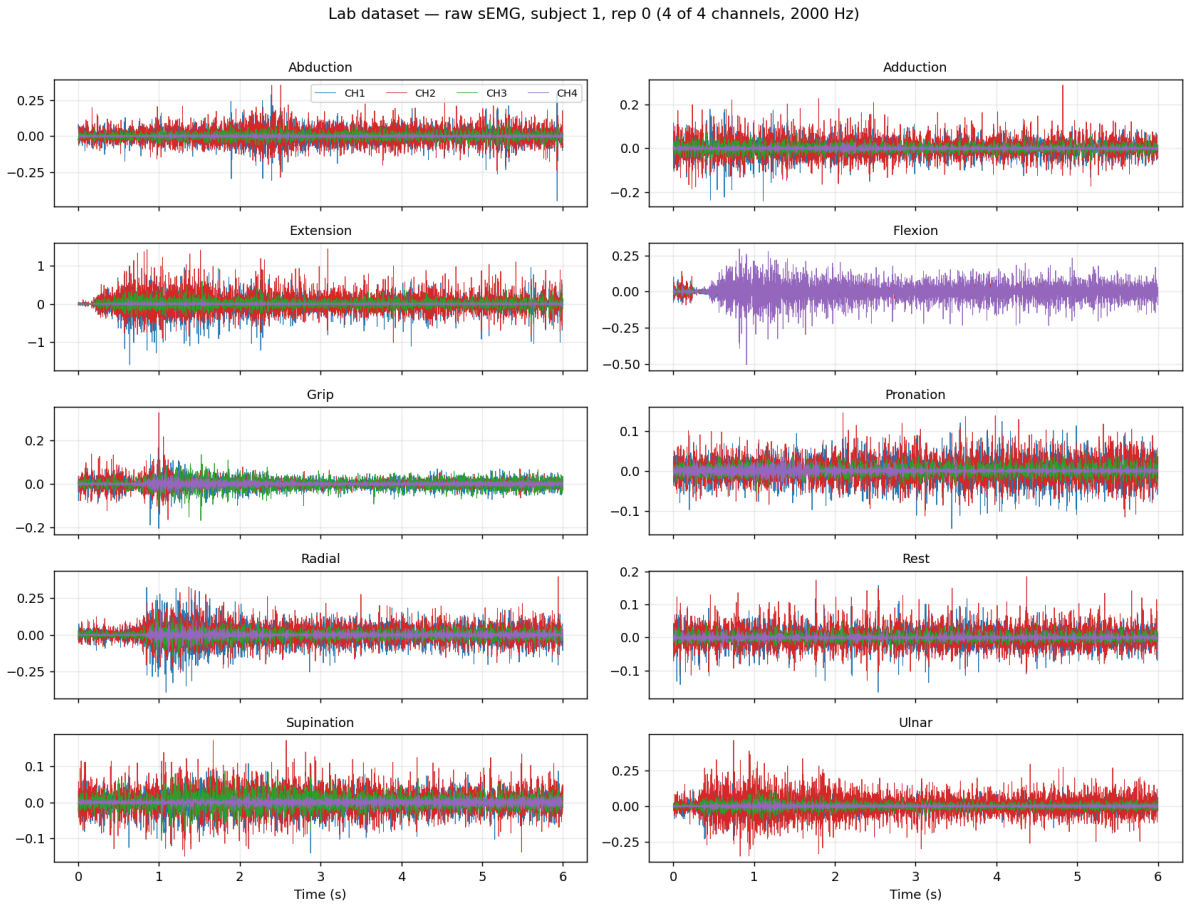


Figure 2 - Lab acquired normalized signals

With preprocessing done, a hybrid feature-extraction stage was applied as shown in table 3.1, combining

3.3 Preprocessing

For every MFCC coefficient, four statistics were computed mean, standard deviation, maximum, and minimum. These were concatenated with the time-domain descriptors into a single feature vector that holds both the spectral and the temporal side of each muscle activation [22].

To make the model less sensitive to electrode shift and muscle fatigue, the training data was augmented artificially time shifts of up to 10% of the signal length, amplitude scaling within 10%, and injected Gaussian noise. This mimics the variability of real recordings and has been reported to improve cross-session robustness noticeably [4].

Following the preprocessing steps, multiple machine learning algorithms were deployed to test and compare against each other, using F1, accuracy, recall, and precision as the main indicators for success due to their relevance in the control algorithm in later chapters.

Table 3 Extracted features for machine learning algorithms

Category	*Features / Parameters*
MFCC Features	13 Mel-Frequency Cepstral Coefficients (MFCCs) per channel
MFCC Window Length	0.025 × fs (50 ms)
MFCC Overlap Length	0.015 × fs (30 ms)
Time-Domain Descriptors (TDD)	Root Squared Zero-Order Moment (RSQ), Sparseness (SP), Irregularity Factor (IF), Root Mean Square (RMS), Waveform Length (WL), Zero Crossing (ZC)

3.3.1 Model Architectures and Hyperparameter Configuration

To evaluate the effectiveness of different machine learning and deep learning approaches for sEMG gesture classification, five models were implemented and compared Random Forest (RF), Support Vector Machine (SVM), Histogram Gradient Boosting Machine (GBM), Compact Recurrent Convolutional Neural Network (CRNN), CNN-BiLSTM with Channel Attention, and an STFT-based 2D Convolutional Neural Network (STFT-2D CNN). Classical machine learning models were implemented using a standard feature-scaling pipeline, whereas deep learning models operated directly on windowed sEMG data.

Table 4 - Model Architectures and Key Hyperparameters

Model	*Architecture / Configuration*
Random Forest (RF)	RandomForestClassifier with 300 decision trees (n_estimators = 300)
Support Vector Machine (SVM)	Radial Basis Function (RBF) kernel, C = 10, gamma = "scale"; training limited to 20,000 windows due to computational complexity
Gradient Boosting Machine (GBM)	HistGradientBoostingClassifier with max_iter = 300
CRNN	Recurrent Convolutional Neural Network configured using dataset-specific channel and class counts
CNN-BiLSTM + Channel Attention	Channel-attention gate (AvgPool → Conv1D 1×1 → Sigmoid), dual convolution branches (kernel sizes 15 and 5, 32 filters each), deep convolution block (Conv1D k=7, 64 filters → Conv1D k=5, 128 filters), MaxPooling, Dropout (0.3), Bidirectional LSTM (128 units), mean pooling, fully connected layers (256 → 128 → K) with Dropout (0.4 and 0.3)
STFT-2D CNN	Short-Time Fourier Transform (Hann window, n_fft = 64, hop length = n_fft/4), log-magnitude spectrogram, dual-path 2D CNN (5×5 and 3×3 kernels, 32 → 64 → 128 filters), Global Average Pooling, fully connected layers (256 → 128 → K)

All classical machine learning models were trained within a StandardScaler preprocessing pipeline to normalize feature distributions prior to classification.

3.3.2 Deep Learning Training Configuration

All deep learning models were trained using the AdamW optimization algorithm. To mitigate class imbalance, class-weighted cross-entropy loss with label smoothing was employed. The best-performing model checkpoint on the validation set was retained for final evaluation.

Table 5 - Deep Learning Training Hyperparameters

Parameter	*Value*
Optimizer	AdamW
Learning Rate	1×10^{-3}
Weight Decay	1×10^{-4}
Number of Epochs	40
Quick-Test Epochs	5
Batch Size	64
Loss Function	Class-weighted Cross-Entropy
Label Smoothing	0.05
Input Normalization	Per-channel standardization using training-set statistics
Checkpoint Selection	Best validation accuracy

3.4 Overview

The methodology was first developed and validated on a laboratory-grade dataset before the lower-cost wearable build. That dataset was recorded from 40 participants across four sEMG channels at 2000 Hz, covering the ten hand and wrist gestures, which gave a balanced set of 2000 trials (200 per class). Several model families were trained and compared on it, ranging from classical feature-based classifiers through to deep convolutional–recurrent networks and a spectrogram-based 2D CNN. The aim was to find which combination of representation and architecture separated the ten gestures most reliably, and to understand how each one behaved when it was asked to generalise to participants it had not seen during training. Each algorithm is described below with a reference to its confusion matrix; the headline results are gathered in Tables 3.5 and 3.6.

3.4.1 Multi-scale CNN-BiLSTM with channel attention

The deep-learning model has three main parts. First is a multi-scale CNN feature extractor, built from parallel convolutional branches with kernel sizes ranging from 15 down to 5 to pick up long- and short-term temporal patterns. The blocks after that use kernel sizes of 7, 5, and 3 with filter depths of 64, 128, 256, and 512 feature maps. Each block carries batch normalisation, ReLU activation, max pooling with stride 2, and dropout to hold back overfitting.

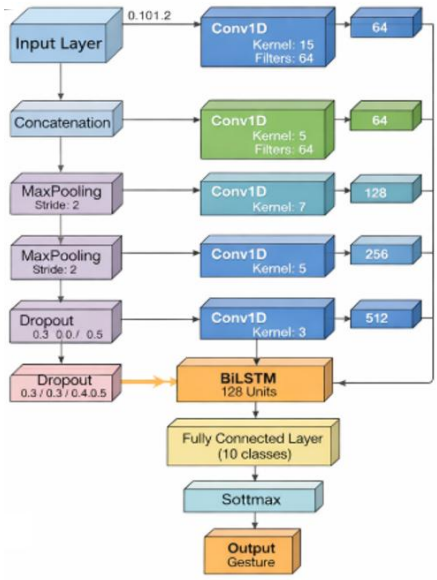


Figure 3 - Proposed Multi-scale CNN-BiLSTM Architecture

Next, a bidirectional LSTM layer with 128 hidden units models how the signal develops forwards and backwards in time, which captures the onset and release of each gesture [29]. The final stage is a fully connected layer with 10 output neurons and a softmax that turns the activations into class probabilities.

The channel attention (CA) mechanism is where the design departs from a conventional CNN. With four channels, not every sensor carries the same information for a given gesture. Radial deviation might show up strongly on one electrode, while rest sits as low-level noise across all of them. The CA layer learns a weight vector that scales each channel's feature maps, quieting the electrodes that mostly carry noise and emphasising the ones with real information. This feature-level denoising lets the BiLSTM focus on the temporal pattern of the muscles that matter. Reading the sequence in both directions, it also picks up the pre-activation phase, the few milliseconds before a movement becomes visible, which is part of how the system reaches the 90.73 ms average latency measured in the experiments. Pairing spatial attention with bidirectional timing gives each of the ten gestures a distinct spatio-temporal signature.

Three splitting schemes were tried. The first was an 80-20 same-subject split, where both training and validation drew from every subject, with four repetitions of each gesture going to training and one or two to validation. The second was a 70-30 split for the cross-subject case, with subjects 1-28 used for training and 12 subjects held out and tested blind, to see how the model handled people it had never seen. The third, and the one finally adopted, was 5-fold cross-validation. Rather than fixing a single split, the data was divided into five equal parts and

the model trained five times, each run leaving one part out for validation, so every signal served as validation exactly once. This gave the most honest picture of the architecture, since sEMG is random enough that one arbitrary split can flatter or punish a model. It also showed whether confusions between classes came from an unlucky split or from something real in the data or the physiology of the movements.

Cross-entropy was used as the loss, which suits multi-class problems. The SoftMax output is a probability distribution over the classes, and cross-entropy measures the logarithmic distance between the predicted distribution and the true label. It was chosen over mean squared error because it produces a large loss when the model is confidently wrong, rather than penalising a wrong answer by a flat amount.

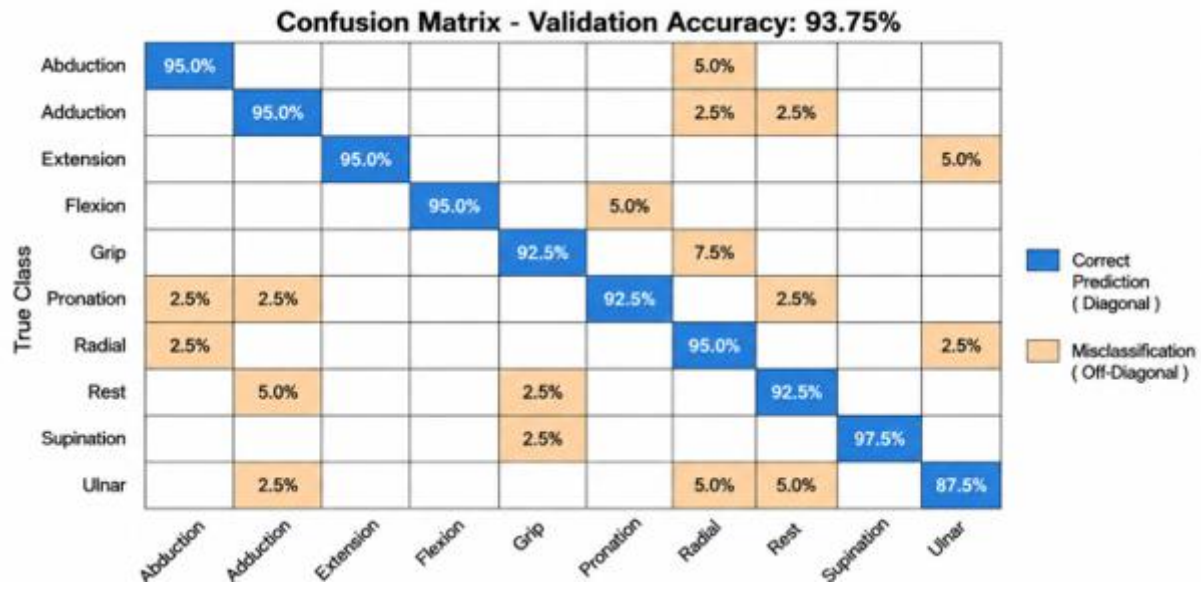


Figure 4 - CNN-BiLSTM Confusion Matrix

The primary architecture was a multi-scale CNN-BiLSTM with a channel-attention stage. Parallel convolutional branches pick up long- and short-term temporal patterns, a bidirectional LSTM models how the activation evolves forwards and backwards in time, and the channel-attention layer weights the four electrodes so that the most informative ones dominate. The model was assessed with 5-fold cross-validation the data was split into five equal parts and the network trained five times, each run holding one part out for validation, so every trial served as validation exactly once. This gave a mean accuracy of 91.80% ± 1.92%, with the folds ranging from 89.25% to 94.25%. Per-class accuracy was strongest for flexion (97.50%) and extension (95.50%) and weakest for rest (82.50% ± 6.37%), with the most common single confusion being rest misread as adduction (12.0%). Cross-validation was the

most realistic estimate available for this architecture, because sEMG is variable enough that any single fixed split can flatter or punish a model.

Because this model is reported as a cross-validation average rather than a single held-out test, the result is a per-fold and per-class accuracy summary rather than one confusion matrix.

To conclude the gestures that were to be picked for the final robotic arm's control the 10 gestures were compared in their relevant metrics.

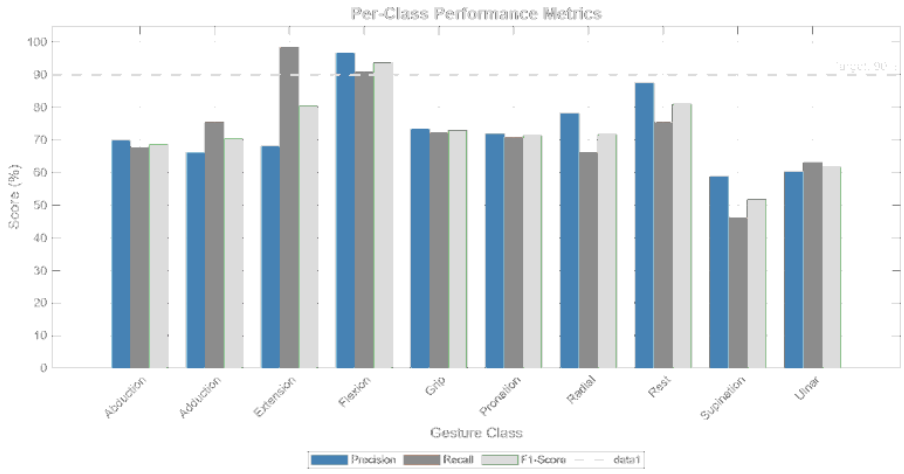


Figure 5 CNN-BiLSTM per-class performance

As can be seen in figure 5, the most reliable pairs were flexion and extension, followed by rest and grip. They were chosen as the main control move groups for the robotic arm, when the data was later collected for the robotic arm, they were kept while the ulnar and radial deviations, as well as pronation, supination, adduction and abduction were changed to easier to classify gestures.

Further inspection and identification of highly confused class pairs was applied to discern the reason; as can be seen in Table 6. These errors come mainly from biomechanics, not from an unstable model. The real difficulty is telling apart gestures that share muscle synergies when only a few electrode channels are available.

3.4.2 Triple-scale CNN + dual BiLSTM with STAE gating

A second, deeper architecture combined three parallel 1D convolutional branches (long, medium, and short kernels) with two stacked bidirectional LSTM layers, preceded by a short-time adaptive energy (STAE) gate that softly attenuates below-threshold background activity.

Where the multi-scale model above was estimated by cross-validation, this network was evaluated on held-out splits to prove how well it generalizes, both within and across participants.

Table 6 Highly confused pairs

True Class	*Predicted As*	*Percentage*
Extension	Ulnar	62.5%
Rest	Pronation	30.0%
Adduction	Rest	27.5%
Adduction	Grip	22.5%
Adduction	Supination	20.0%
Abduction	Radial	17.5%
Adduction	Pronation	12.5%
Radial	Ulnar	12.5%
Grip	Supination	10.0%
Radial	Grip	10.0%

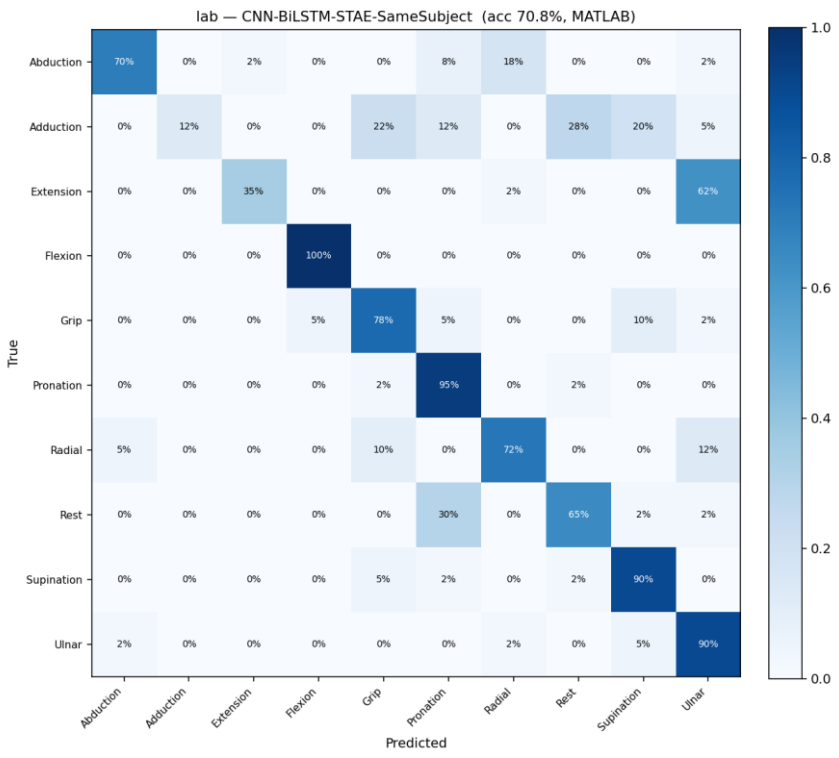


Figure 6 CNN-BiLSTM confusion matrix for lab acquired data

Testing was performed and yielded the following results, where it shows in Figure 5 that flexion and extension are the leading performers due to the unique nature of their inter-muscle activation. In a same-subject 80–20 split, where every participant appeared in both training and validation, the model reached 70.75% accuracy.

Metric	***Result***
Accuracy (%)	70.75
Macro Precision (%)	77.14
Macro Recall (%)	70.75
Macro F1-Score (%)	68.02
Average Latency (ms)	90.73

In a different-subject 80–20 split, where the validation participants were held out entirely, it reached 72.62% accuracy (macro precision 73.10%, macro recall 72.62%, macro F1 72.36%) at 148.46 ms (see Figure 6). The close, slightly higher cross-subject figure is encouraging, since it suggests the STAE-gated model was learning gesture structure rather than memorizing individuals.

3.4.3 STFT spectrogram with dual path 2D CNN

The final deep approach treated the problem as an image-classification task. Each trial was turned into a log-magnitude short-time Fourier transform (STFT) spectrogram and fed to a dual-path 2D CNN, with one path using larger 5×5 kernels for long-term spectral structure and the other 3×3 kernels for fine detail, their global-average-pooled outputs fused before the classifier. It was trained with no recurrent layers, relying purely on spatial and spectral features. Under the unified comparison on the laboratory data it reached 75.5% accuracy (macro F1 0.753), its confusion matrix shows the same broad pattern as the other models — strong wrist flexion/extension and rest, weaker low-amplitude rotations (see Figure 7).

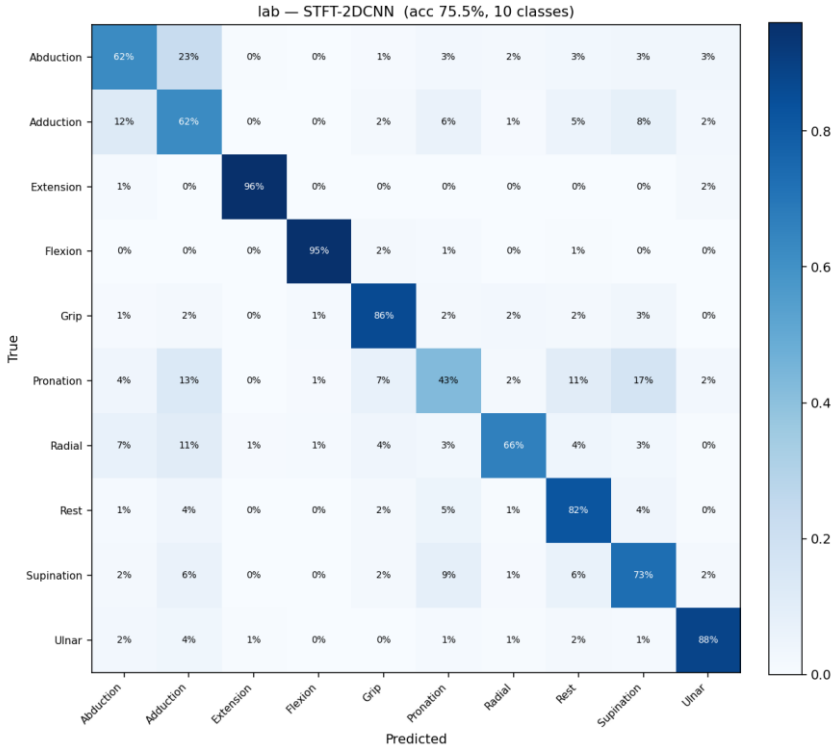


Figure 7 - STFT 2D-CNN confusion matrix for lab acquired data

3.4.4 Random Forest

A random forest was trained as a classical baseline on the same per-window MFCC + time-domain feature vectors. It builds an ensemble of decision trees, each grown on a bootstrap sample of the data and a random subset of features at every split and classified by majority vote across the trees; the ensemble averages out the variance of any single tree and rarely overfits. The laboratory data was the strongest of the classical models, reaching 74.3% accuracy (macro precision 75.0%, macro recall 74.3%, macro F1 0.744). Its confusion matrix (Figure 8) concentrates on the diagonal for the high-amplitude gestures while spreading across the low-amplitude wrist and rotation classes, the same biomechanical confusion seen in the deep models.

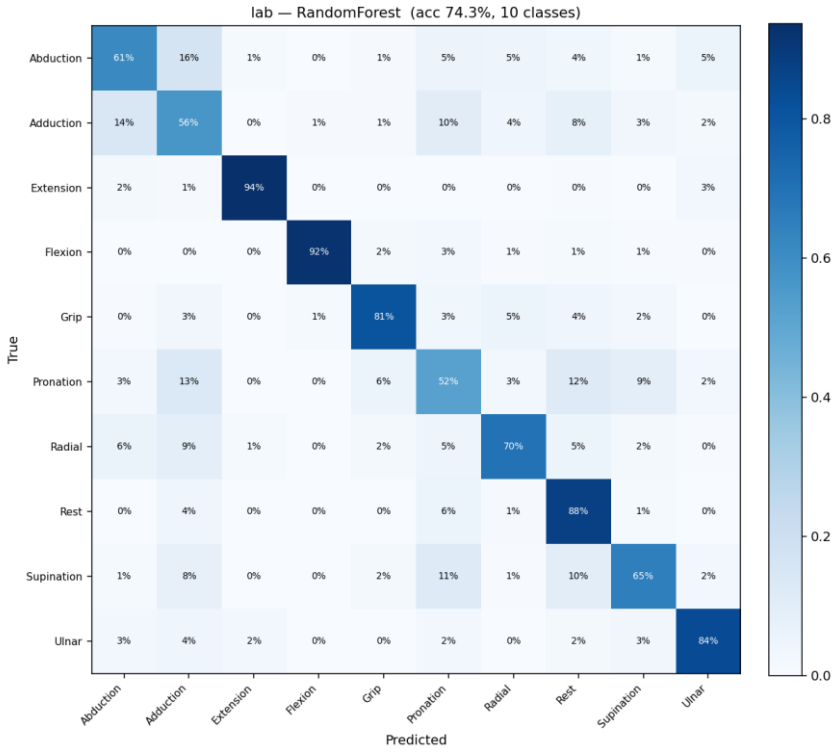


Figure 8 - RF Confusion matrix for lab acquired data

3.4.5 Support Vector Machine (SVM)

A support vector machine with a radial-basis-function (RBF) kernel was trained on the same feature vectors. The SVM finds the decision boundary that maximizes the margin between classes, and the RBF kernel lets that boundary be non-linear by implicitly mapping the features into a higher-dimensional space; it is a strong classifier for moderate feature dimensions but scales poorly to very large training sets. On the laboratory data it reached 71.0% accuracy (macro precision 71.4%, macro recall 71.0%, macro F1 0.710), sitting just below the random forest. Its confusion matrix is shown in Figure 9.

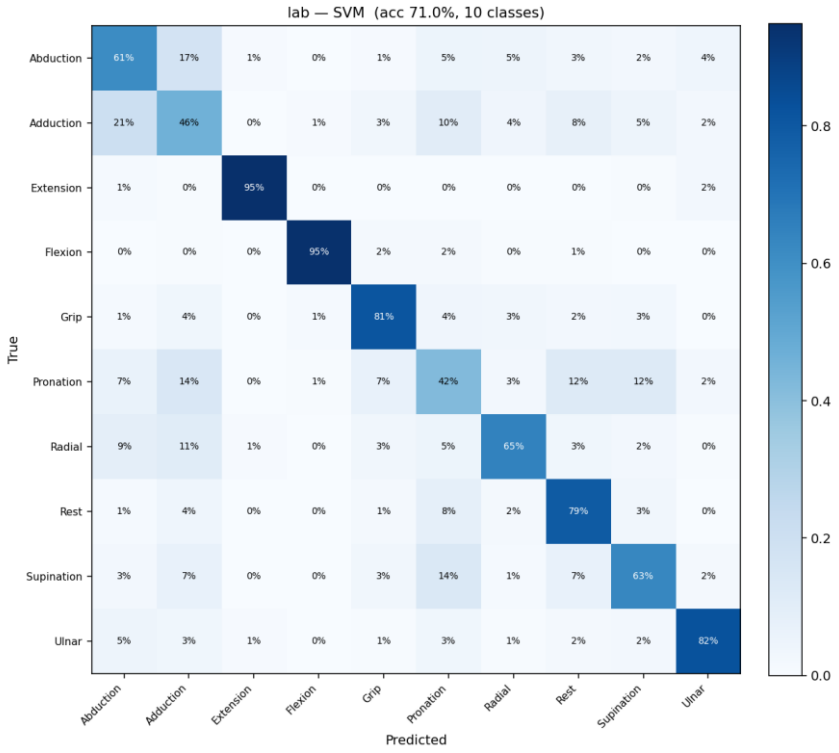


Figure 9 - SVM Confusion matrix for lab acquired data

3.4.6 Gradient Boosting

A gradient-boosting classifier was trained as the third classical baseline. Unlike the random forest, which grows trees independently and votes, gradient boosting builds trees sequentially, each one fitting the residual errors of the trees before it, so the ensemble is corrected stage by stage. It is often very accurate but is the slowest of the classical models to train. On the laboratory data it reached **68.9%** accuracy (macro precision 69.3%, macro recall 68.9%, macro (F1 0.689), the lowest of the three classical baselines but still far above chance. Its confusion matrix is shown in Figure 10.

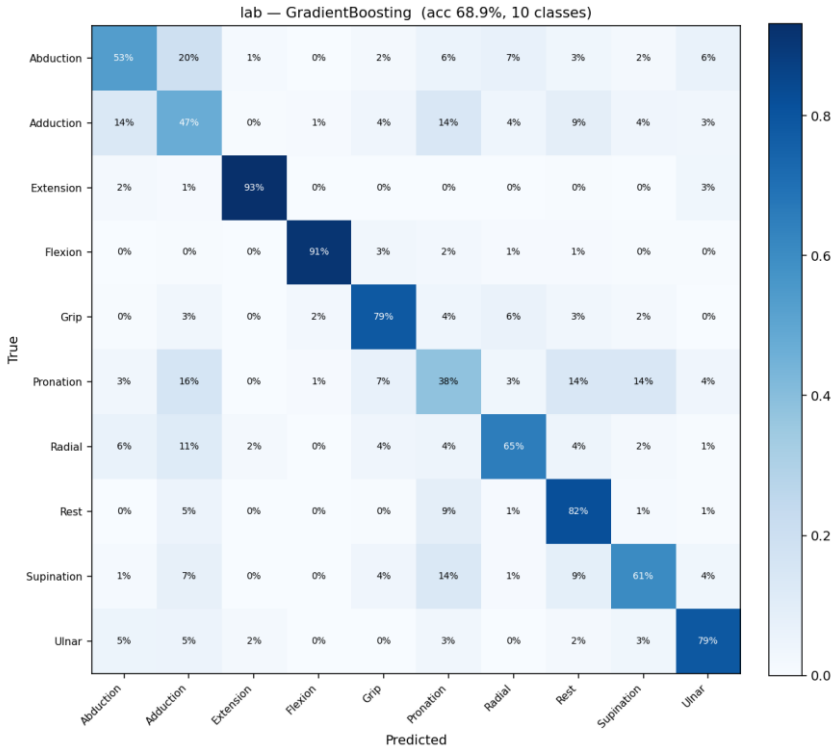


Figure 10 - GBM confusion matrix for lab acquired data

3.4.7 Recurrent CNN (CRNN)

compact recurrent convolutional network was also evaluated. Rather than the summary feature vector, it consumes the raw window directly a short 1D convolutional stack learns local temporal shapes, a bidirectional GRU then models how those activations evolve across the window, and a linear head produces the class scores (≈ 99,500 parameters). On the laboratory data it reached 72.0% accuracy (macro precision 72.3%, macro recall 72.0%, macro F1 0.718), comparable to the SVM. Its confusion matrix is shown in Figure 11.

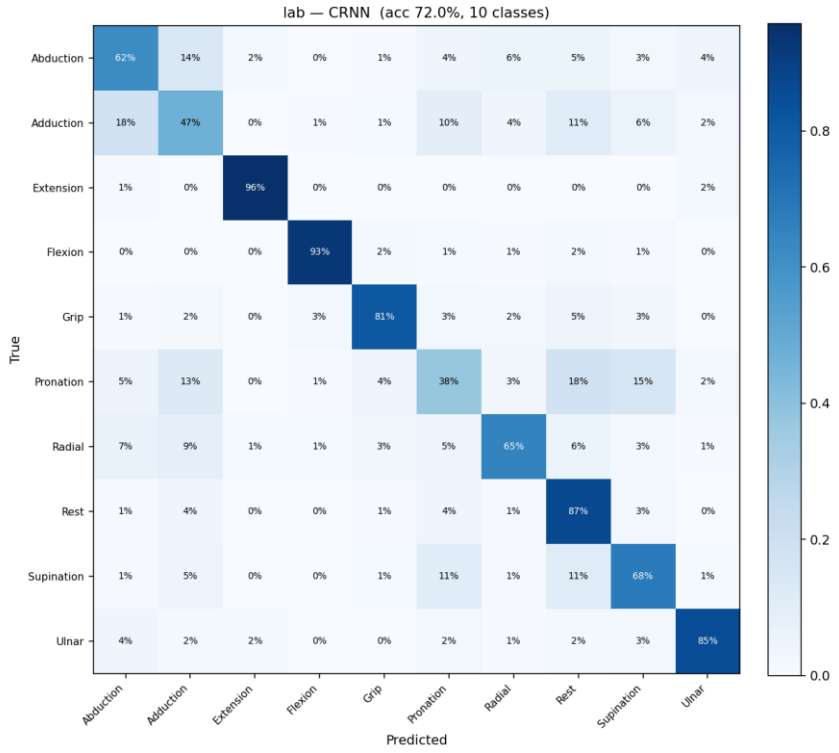


Figure 11 - CRNN confusion matrix for lab acquired data

3.5 Post-Processing and resultant evaluation

The training and evaluation procedures were performed using a cross-cycle validation strategy to assess generalization across recording sessions. GPU acceleration was automatically utilized for deep learning models when available, whereas classical machine learning models were executed on the CPU. The final deployed Random Forest model achieved the strongest performance under the cross-cycle evaluation protocol. On the laboratory dataset consisting of ten gesture classes acquired from four sEMG channels, the model obtained an overall

classification accuracy of 74.3% and a macro F1-score of 0.744, demonstrating robust performance under realistic recording conditions.

Adduction was confused with several classes, extension with ulnar deviation, and rest with pronation, and each of these has a clear physiological reason. The confused pairs share muscles such as the extensor carpi ulnaris, the flexor digitorum, and the pronator teres. Their activation profiles and spectral content overlap, something well documented in the literature, particularly under low-force contractions. The high conditional probability of misclassification says that the variation within a class is as large as, or larger than, the variation between classes, so the feature distributions cluster together. This matches earlier work showing that physiologically similar gestures sit in neighbouring regions of the feature space [30][14][6].

In short, the confusion structure tracks anatomy rather than any flaw in training.

To deal with these state misclassifications, post-processing decision logic was layered on top of the network output. Gesture execution was allowed only for predictions above a confidence threshold, and the misclassification patterns were analysed directly. Class weighting during training pulled the model away from favouring high-activity gestures [31] and improved the split between rest and low-amplitude movements [32].

The network output fed a dedicated inference node, which published the recognised gestures to the robotic control framework built on ROS2 Humble. A command-listener node mapped each gesture to a predefined joint trajectory through the MoveGroup interface in MoveIt2. Joint limits, velocity scaling, and collision avoidance were set so the arm would operate safely, and a small state machine blocked unstable transitions between gestures [33].

The final framework has four layers, each grounded in studies tied to the project's aims. Feature extraction uses a hybrid scheme 13 Mel-Frequency Cepstral Coefficients per channel together with six time-domain descriptors, namely Root Squared Zero Order Moment, Sparseness, Irregularity Factor, Root Mean Square, Waveform Length, and Zero Crossing. Each MFCC coefficient contributes four statistics (mean, standard deviation, maximum, minimum), which gives a 116-element feature vector per trial. Data acquisition uses sEMG sensors placed over the main forearm extensor and flexor groups, following SENIAM placement guidelines [21]. An ESP32-S3 microcontroller samples the signals at 500 Hz and sends them to a host computer for recording. The gesture set is the same ten hand and wrist movements rest, grip, flexion,

extension, radial deviation, ulnar deviation, supination, pronation, abduction, and adduction, each recorded under controlled conditions with visual cues.

The recognised gesture is published by a ROS2 Jazzy inference node and mapped to a predefined joint trajectory by a MoveIt2 command-listener node through the MoveGroup interface. A state machine blocks unstable transitions and applies confidence thresholding in post-processing to cut down false rest-state activations (Barona Lopez et al., 2024). The methodology brings together the following methods

Hardware design and integration a custom 6-DOF arm designed in SolidWorks and 3D-printed in ABS by FDM, with a URDF exported for ROS2 and MoveIt2. Computer simulation Gazebo physics simulation using the mass-accurate URDF to check trajectories before anything ran on hardware. Experimental validation testing the physical prototype under sEMG control to confirm the system works end to end.

Chapter (4) Experimental Work

This chapter reports how the complete sEMG-to-arm system was built and evaluated. It keeps the analytical and simulation work, used to predict and check the system's behaviour, separate from the physical prototyping and the end-to-end gesture-control loop. Where the as-built system differs from the design described earlier, the deployed configuration is what is reported here and treated as authoritative. The work followed a staged plan. The manipulator and its electronics were designed and built first, then the acquisition chain was checked on the bench. Gesture data was collected next; features were extracted, and several classifiers were trained and compared before the strongest model was carried forward to drive the arm. Each stage fed the next, and evaluation was kept separate from training so that the reported accuracy reflects unseen data rather than memorised examples. Together, the mechanical, electrical and control designs define a complete path from forearm muscle activity to coordinated joint motion. The chapter that follows describes how this design was exercised experimentally, what the recorded data revealed, and how well the trained models performed.

4.1 Mechanical Design

This part of the chapter covers the mechanical design of the custom six-degree-of-freedom (6-DOF) arm that serves as the physical platform for the work. The process ran from defining requirements, through CAD modelling in SolidWorks, to exporting a URDF for use with ROS2 and MoveIt2. What follows is the conceptual design, then the detailed design, the main design challenges and how they were solved, and finally what the design achieved, where it falls short, and how it ties back to the rest of the project.

4.1.1 Introduction

The mechanical platform sets the terms for everything else, since its geometry fixes the workspace, the payload it can carry, and which poses are reachable at all. The goal was a 6-DOF arm that could carry out motions matching the ten classified sEMG gestures, print on an FDM machine, run on servo motors, and export to a URDF for ROS2 Jazzy and MoveIt2 [18].

Four objectives shaped the design. It needed six independent degrees of freedom to cover the reach and orientation range of a human forearm and wrist. It had to print as a modular assembly in ABS. It needed a gripper end-effector that could stand in for the grip gesture. And it had to produce a mass-accurate, inertia-complete URDF for motion planning and simulation. The

main technical hurdle was the top-heavy mass distribution, which forced a dual-servo arrangement at the shoulder elevation joint; the two servos are driven by mirrored, opposite-direction commands so they stay synchronised.

4.1.2 Conceptual Design

Six degrees of freedom was the minimum for full three-dimensional reach with independent control of end-effector orientation, matching the shoulder, elbow, and wrist groupings of the human arm (Craig, 2005). The arm had to print on a standard FDM machine without giving up structural integrity, with all main structural parts in ABS (Dizon et al., 2018). It had to stay modular so a single link could be swapped out, and its total mass had to fit within the servo torque ratings. Finally, the geometry had to export as a mass-accurate URDF, complete with inertia tensors, for Gazebo simulation and MoveIt2 planning [18].

Two routes were weighed. The first was buying an off-the-shelf 6-DOF platform. Commercial arms come with validated kinematics and sometimes ROS drivers [17], but they rarely expose per-link inertial parameters in URDF form, their joint limits are fixed and may not line up with the sEMG gesture set, and they cannot be reworked for sensor mounting. A commercial 6-DOF arm with ROS2 support was also judged too expensive.

The second route, and the one chosen, was a custom design built from scratch in SolidWorks and printed in ABS by FDM. This gave full control over link sizes, joint placement, and servo housings, so the CAD model and the physical prototype matched closely. More importantly, it allowed every link to be characterised inertially through SolidWorks mass-property analysis, which a physically accurate URDF depends on. Assigning ABS density to each part and reading the mass and inertia tensor straight out of the CAD environment meant the simulation model carried the real dynamic properties of the printed arm.

4.1.3 Detailed Design

The arm is built from eight structural links in a serial chain with six active joints, summarised in Table 3.1. The base link (base_link) is the fixed mount. Links l1 to l5 make up the main arm segments, and l6r and l6l are the right and left fingers of a parallel gripper. Joint j1 is a continuous joint that swivels the whole arm horizontally. Joints j2 to j5 are revolute. j2 drives shoulder elevation, and the gripper at j6 uses a mimic joint with a ratio of −2.0885, so a single servo input closes both fingers symmetrically.

Table 7 Joint Specifications of the 6-DOF Robotic Arm

Joint Name	*Type*	*Axis*	*Description*
Joint 1	Continuous	X	Base rotation
Joint 2	Revolute	Z	Shoulder elevation
Joint 3	Revolute	Z	Elbow flexion
Joint 4	Revolute	X	Wrist roll
Joint 5	Revolute	Z	Wrist pitch
Joint 6	Revolute	Y	Gripper actuation
Joint 6 (Mimic)	Revolute (Mimic)	—	Synchronized gripper finger motion (−2.4× multiplier)

The full assembly is shown in Figures 12. Standing in its neutral upright pose, the arm is 670.24 mm tall on a circular base 150.00 mm across. Its height splits into four main vertical segments 247.50 mm at the bottom, 172.47 mm through the mid-arm, 107.50 mm in the upper section, and 74.75 mm for the wrist and gripper. The base plate is 89.00 mm wide at its mounting.

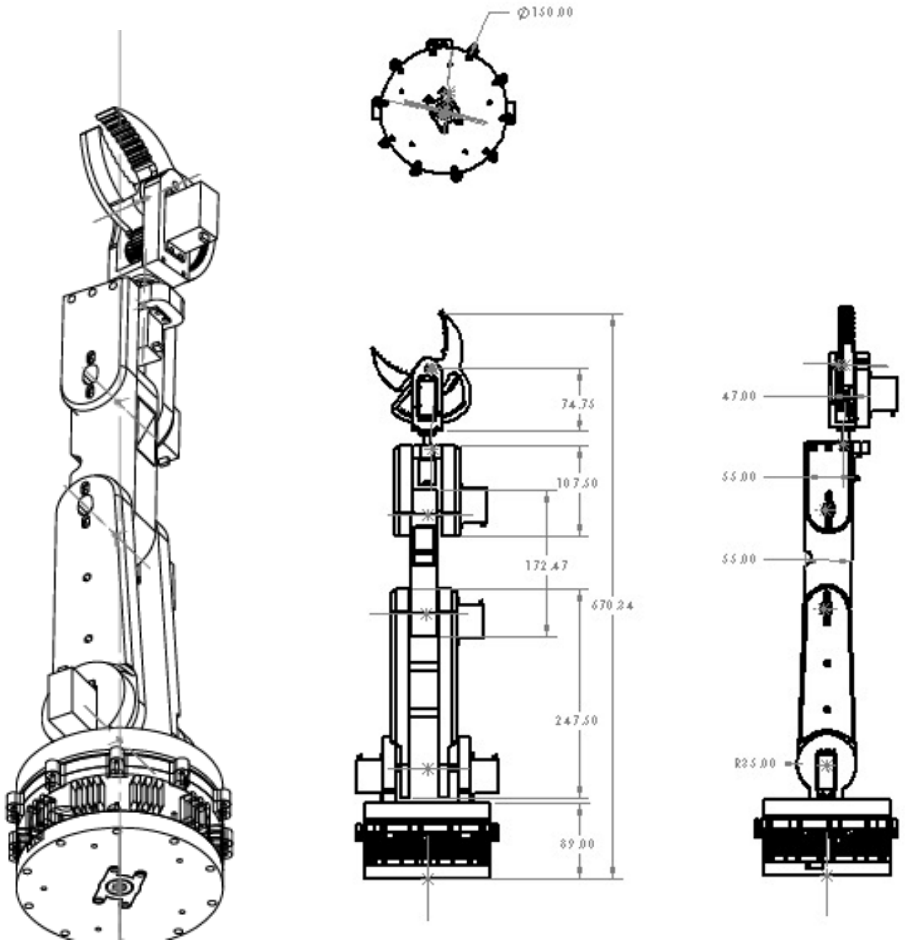


Figure 12 Dimensioned engineering drawing

Every structural link was printed in ABS by FDM. ABS was picked for its higher glass-transition temperature, around 105°C against roughly 60°C for PLA, which helps at the servo mounting points where heat builds up, and for its better impact resistance under the repeated loading of gesture cycles (Dizon et al., 2018). SolidWorks computed the mass of each link, servo masses included, from the assigned ABS density. The assembled arm weighs 2.41 kg. Table 3.2 lists the link masses and their principal inertia terms.

Table 8 Link Mass Properties

LINK NAME	*Mass (kg)*	*Ixx (kg·m²)*	*Izz (kg·m²)*
Base Link	0.6126	1.065×10^{-3}	2.049×10^{-3}
Link 1	0.4309	9.315×10^{-4}	5.532×10^{-4}
Link 2	0.5379	1.924×10^{-4}	1.982×10^{-3}
Link 3	0.2595	5.712×10^{-4}	6.121×10^{-4}
Link 4	0.4003	3.537×10^{-4}	1.153×10^{-4}
Link 5	0.1118	5.063×10^{-5}	1.567×10^{-5}
Link 6 (Right)	0.0316	4.020×10^{-5}	9.679×10^{-6}
Link 6 (Left)	0.0252	1.289×10^{-5}	1.042×10^{-5}
Total	**2.4098**	—	—

The SolidWorks assembly was exported to URDF with the sw_urdf_exporter plug-in (v1.6.0-4-g7f85cfe; Brawner, http //wiki.ros.org/sw_urdf_exporter). Before export, a coordinate frame was placed at each joint origin with its axes aligned to the relevant plane of motion. The URDF was then checked in RViz2 and Gazebo under ROS2 Jazzy, which confirmed the inertia tensors, joint axes, and link geometry were consistent. The gripper's mimic joint was written explicitly into the URDF so the symmetric finger closure could be simulated [18].

The main design problem was the arm's mass distribution. Everything above the shoulder joint j2, links and servos together, comes to about 1.37 kg. At raised joint angles that mass produces a large gravitational moment about j2, more than a single servo's rated stall torque can hold. The fix was a dual-servo arrangement at j2 two MG995 units coupled to the same joint output,

sharing the load and giving a combined stall torque of about 2.94 N·m. Because the two servos sit on opposite sides of the shoulder axis, they have to turn in opposite directions to make one coherent motion. The software handles this by sending them mirrored, equal-magnitude angle commands of opposite sign, both derived from the same target joint angle, so the horns stay symmetric and the two motors counter-rotate to share the cantilever load across the whole j2 range.

4.1.4 Discussion and Conclusion

The mechanical design met its main objectives. The arm has six independent degrees of freedom the turntable swivel (j1), shoulder elevation (j2), elbow flexion (j3), wrist roll (j4), wrist pitch (j5), and gripper actuation (j6). Building it in ABS with modular links means any one link can be replaced without taking the whole arm apart. The parallel claw gripper maps directly onto the sEMG grip gesture. And the SolidWorks-to-URDF workflow produced a simulation-ready model, verified in Gazebo, whose link inertias match the SolidWorks mass-property outputs (Brawner, sw_urdf_exporter).

The mechanical design is what makes the system-level goal possible mapping classified sEMG gestures to joint trajectories on a real 6-DOF arm. Because the mass and inertia properties come straight from SolidWorks, the Gazebo simulation reflects how the physical prototype actually behaves. In that sense the quality of the mechanical design caps the fidelity of the whole control chain, from sEMG inference through to actuation.

4.2 Electrical and Electronics Design

A few limitations remain. First, the dual-servo shoulder at j2 makes the system sensitive to how well the two units match both are the same MG995 model and are kept in step in software through mirrored, opposite-sign angle commands rather than any analogue filtering, but residual differences in gear backlash and centring still have to be trimmed when a servo is replaced. Second, ABS does not hold up to fatigue over long deployments; metal brackets at the high-stress joints (j2 and j3) would extend service life. Third, the URDF joint limits were not constrained during the SolidWorks export and have to be enforced in the MoveIt2 configuration instead. Fourth, the 670 mm height puts the end-effector near the edge of the base torque envelope at full extension. Finally, late in development the base turntable (j1) was changed from a PWM servo to a stepper motor driven by a DRV8825 module, covered in the electrical design below and in Chapter 4; the simulation still treats j1 as a position-controlled

joint, so turning the base command into step and direction pulses on the physical arm is still an open integration point.

4.2.1 Introduction

The rest of this section is laid out in three parts the design methodology and system architecture, the detailed design of each electronic subsystem, and a closing discussion of how well the design works, where it is limited, and how it connects to the wider project.

The electrical and electronics design covers two tightly linked subsystems the sEMG acquisition front end and the arm's actuation back end. Between them they form the hardware bridge from the operator to the manipulator, turning muscle activity into joint motion. The acquisition side captures rectified EMG envelopes from two complementary sensors, a Muscle Sensor v3 and a CHEEZ.semg module, through an ESP32-S3-N16R8 microcontroller. The actuation side drives six MG995 servos and a stepper-driven base turntable through a PCA9685 PWM driver and a DRV8825 stepper driver, all under a Raspberry Pi 4 Model B running ROS2 Jazzy. Signal quality at the front end sets how well the gesture classes can be told apart, and the latency through the actuation path decides whether a recognised gesture turns into timely, correct motion.

4.2.2 Design Methodology

The two stages talk to each other over Wi-Fi through a TCP socket, so the system can run untethered during both data collection and real-time control. The acquisition firmware was written in the Arduino framework for the ESP32-S3, and the voltage-divider resistor values were worked out analytically to keep the sensor output safely inside the ESP32-S3 GPIO range. The servo control software uses the Adafruit PCA9685 Python library on the Raspberry Pi 4. The whole chain was checked end to end, from acquisition through classification to servo actuation.

The electronics form a two-stage pipeline joined over a local wireless link. The first stage handles sEMG acquisition the two sensors (a Muscle Sensor v3 and a CHEEZ.semg module), their passive support circuitry, and the ESP32-S3-N16R8. The second stage handles actuation and inference the Raspberry Pi 4, the PCA9685 PWM driver, the DRV8825 base stepper driver, and the motors. The split is deliberate. The ESP32-S3 does nothing but acquire and transmit, while inference, motion planning, and actuation all run on the Raspberry Pi 4. That keeps the heavy ROS2 Jazzy and MoveIt2 stack on a full Linux machine without loading the acquisition microcontroller with real-time work.

4.2.3 Detailed Design

Table 9 Muscle Sensor v3 Specifications

Parameter	***Minimum***	***Typical***	***Maximum***
Power Supply Voltage (Vs)	±3.5 V	±5 V	±18 V
Gain Setting, Gain = 207 × (X / 1 kΩ)	0.01 Ω (0.002×)	50 kΩ (10,350×)	100 kΩ (20,700×)
Output Signal Voltage (Rectified & Smoothed)	0 V	—	+Vs
Differential Input Voltage	0 mV	2–5 mV	+Vs/Gain

The two sEMG channels use two different analogue front ends, chosen to get a clean envelope from each muscle group without the redundancy of running two identical boards. Channel 1 is an Advancer Technologies Muscle Sensor v3, a self-contained front end that amplifies the differential signal, bandpass filters it, full wave rectifies it, and smooths it, presenting a rectified envelope (always ≥ 0 V) at its SIG pin. It uses three leads RED at the centre of the active muscle belly, BLUE at the distal end, and BLACK on a bony landmark for common-mode rejection, with a ±9 V supply from two 9 V batteries. Channel 2 is a CHEEZ.semg module, a MyoWare-style wet-electrode front end that produces a bandpassed envelope on-device from a 5 V supply and also exposes a digital electrode-contact (wear-detect) line. The Muscle Sensor v3 sits over the flexor carpi radialis on the anterior forearm and the CHEEZ.semg over the extensor digitorum on the posterior forearm, following SENIAM placement guidelines (Barbero et al., 2012). Both channels are sampled at 500 Hz, the CHEEZ module's native envelope rate. Table 4.1 lists the key Muscle Sensor v3 specifications.

The ESP32-S3-N16R8 is a dual-core 32-bit Xtensa LX7 microcontroller with built-in Wi-Fi, 16 MB of flash, and 8 MB of octal-SPI PSRAM. That PSRAM bus permanently takes GPIO33 to GPIO37, so those pins cannot be used as ADC inputs, and ADC2 (GPIO11 to GPIO20) is unreliable while Wi-Fi is active, which leaves ADC1 (GPIO1 to GPIO10) as the only safe option. The chosen pins are GPIO3 (ADC1_CH2) for the Muscle Sensor v3 and GPIO7 (ADC1_CH6) for the CHEEZ.semg signal, with GPIO6 reading the CHEEZ digital wear-detect line; none of these clashes with bootstrapping or the PSRAM bus. The ADC runs at 12-bit resolution with ADC_11db attenuation (a 0–3.9 V input range). Sampling is driven at 500 Hz by a micros()-based spin-wait loop on a 2 ms period. In data-collection mode the two-

channel stream goes over USB-Serial at 921 600 baud to a Windows host; in real-time inference mode the same 500 Hz stream is sent to the Raspberry Pi 4, or a PC, over a Wi-Fi TCP socket, with the ESP32 acting either as its own access point or as a station on an existing network.

Six of the arm's seven actuators are MG995 metal-gear servos running at 6 V, with a stall torque of about 1.47 N·m (roughly 15 kg·cm), a no-load speed of about 8.05 rad/s (roughly 0.13 s per 60°), and a mass of 55 g each. They drive joints j2 to j5 and the gripper, with the shoulder j2 using two of them, so the dual-servo shoulder provides a combined stall torque of about 2.94 N·m. The seventh actuator, the base turntable j1, is a stepper motor driven by a DRV8825 module rather than an MG995, as described below. The servos are controlled with the usual 50 Hz PWM signal and a 1–2 ms pulse width. Table 4.2 gives the full specifications.

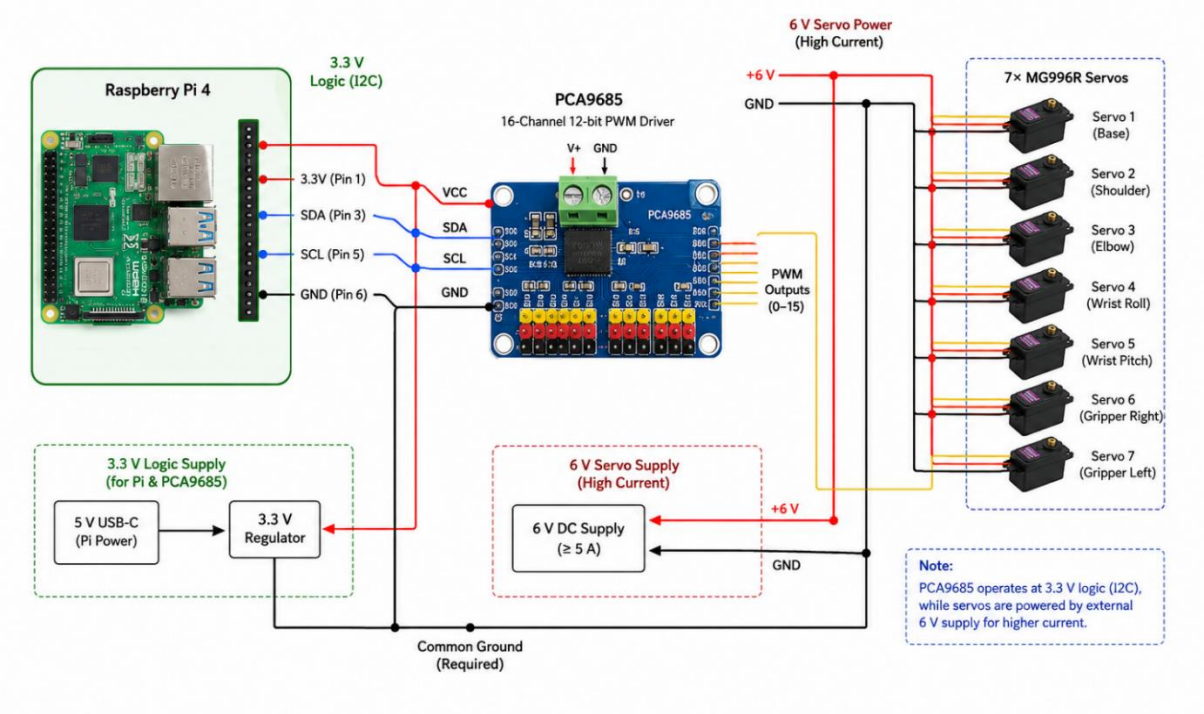


Figure 13 PCA9865 Wiring diagram

The Raspberry Pi 4 Model B runs Ubuntu 24.04 LTS with ROS2 Jazzy and MoveIt2. Its 1.8 GHz quad-core ARM Cortex-A72 and up to 8 GB of LPDDR4 are enough to run the sEMG receiver node, the CNN-BiLSTM inference node, and the MoveIt2 planner at the same time. Three ROS2 nodes run on the Pi the sEMG receiver node takes in live samples over Wi-Fi TCP; the inference node classifies the incoming stream with the trained model; and the trajectory node maps gesture commands to joint trajectories through the MoveGroup interface in MoveIt2, passing them to the PCA9685 via the ros2_control hardware interface [18].

4.2.4 Discussion and Conclusion

The electronic design carries the signal all the way from muscle to joint actuation. The acquisition circuit delivers a clean signal range at the ADC inputs, which makes reliable 12-bit quantisation at 500 Hz possible without saturation. The wireless TCP link removes the USB tether, so the wrist can move freely during both data collection and real-time control. The PCA9685 generates precise multi-channel PWM off a single I2C bus, and the dual-servo j2 produces smooth, load-sharing shoulder elevation through the mirrored opposite-direction commands.

Several limitations stand out. First, both sensors output a rectified, smoothed envelope rather than raw bipolar EMG, which strips out the high-frequency content and narrows the feature space compared with research-grade hardware. Second, as the experiments later confirmed (Chapter 4), the CHEEZ.semg channel did not couple properly to the ADC during the recorded sessions and gave almost no usable signal, leaving the system effectively single channel; this is a hardware and wiring fault that caps the accuracy that can be reached. Third, the Muscle Sensor v3 envelope does nothing to suppress 50 Hz mains interference, which shows up as a stationary pedestal in its spectrum and is dealt with by a software notch filter described in Chapter 4. Fourth, the peak servo current demand calls for a properly rated dedicated supply, or the rail sags and the servos behave erratically under high load.

The electronic design feeds the research objective directly. The 500 Hz rectified-envelope acquisition supplies the windowed feature extraction and classification described later, and the PCA9685–MG995 actuation chain, with the DRV8825-driven base alongside it, is the physical layer that carries out the gesture-to-joint commands. Together they close the loop from sEMG classification to physical manipulation.

4.3 Control Design and Implementation

The system runs two parallel control paths that share the same operator conventions but talk to different back ends. The simulation path is built on ROS 2 it publishes JointTrajectory messages to ros2_control JointTrajectoryControllers, run inside Gazebo by the gz_ros2_control plugin. The hardware path is plain Python with no ROS dependency it writes servo commands through the Adafruit ServoKit library to a PCA9685 PWM driver over I²C, which drives the physical MG995 servos. A deliberate choice keeps the two paths in physically separate folders with no shared runtime dependency, so the hardware path imports neither ROS nor the simulation package and can simply be copied onto the Raspberry Pi and run on its own.

What they do share is the operator-facing contract, so control skill built up in simulation transfers straight to hardware, and that same contract becomes the integration seam for the sEMG mode. Figure 14 shows the architecture.

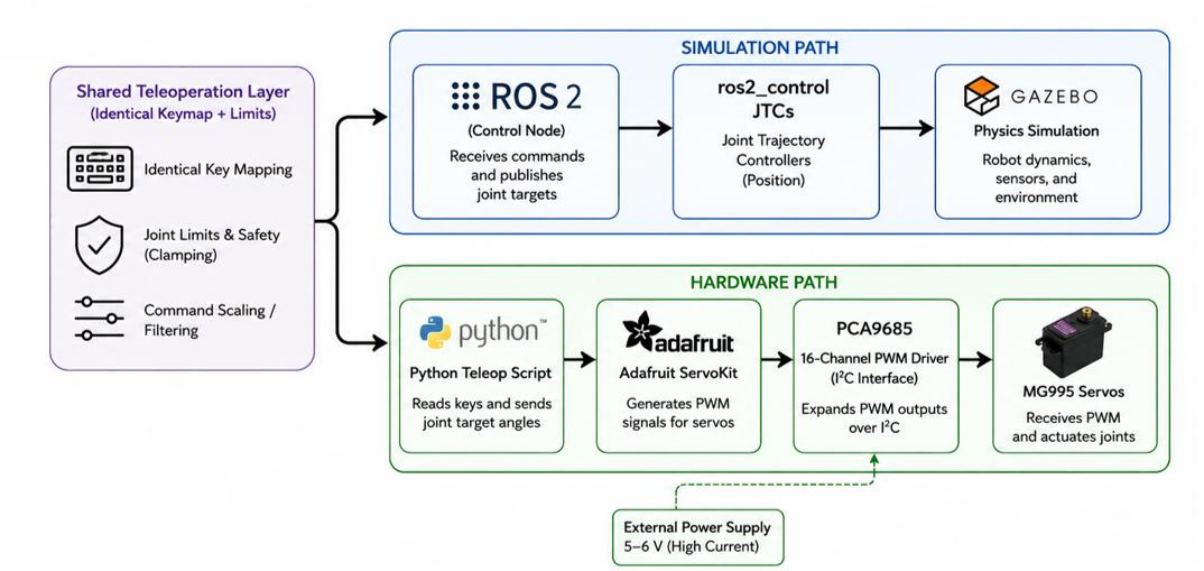


Figure 14 - Control architecture block diagram

On the physical arm there is no ROS and no trajectory controller. One Python program reads the keyboard, holds a target angle for every joint, and writes servo commands to the PCA9685 over I²C; the PCA9685 generates the 50 Hz PWM signals the MG995 servos need, taking the precise pulse timing off the Raspberry Pi's CPU. The software keeps joint state in radians, matching the URDF, and converts to servo degrees only at the output. For a joint commanded to angle θ (radians), each servo channel c is driven to servo_deg(c) = center_deg(c) + sign(c) · k(c) · θ, where center_deg(c) is the servo angle that puts the joint at its mechanical zero pose (nominally 90°, the mid-travel point, refined by calibration), sign(c) ∈ {+1, −1} sets the direction, and k(c) is the gain in servo-degrees per radian (about 57.3°/rad for a direct 1 1 drive). The result is clamped to the servo's physical travel range, [0°, 180°].

The dual-servo shoulder j2 is written in software as two channels with opposite signs and equal gain servo_deg(ch1) = 90° + (+1)·k·θ and servo_deg(ch2) = 90° + (−1)·k·θ. The two horns therefore stay symmetric about their centres, so for any joint angle θ the displacements (ch1 − 90°) and (ch2 − 90°) are equal and opposite. A single operator command moves the shoulder as one rigid joint while the two motors physically counter-rotate to share the load. The scheme is purely feed-forward, with no inter-servo feedback or speed-matching filter, because both servos get geometrically consistent targets from the same commanded joint angle.

The hardware control software is kept deliberately small, layered, and self-contained, in the raspberry_pi/arm_control package; its modules are listed in Table 5.3. The key feature of the ArmDriver abstraction is that it picks one of two back ends when it is constructed a hardware back end that talks to the real PCA9685 through the adafruit-circuitpython-servokit library, or a mock back end that prints the commands it would send and moves nothing. The mock back end kicks in automatically when the Adafruit libraries or the I²C bus are missing, or on demand through a --dry-run flag. That let the whole control logic, the radian-to-degree mapping, the mirrored-shoulder symmetry, the joint-limit clamping, and the gripper stepping, be exercised on a development PC with no hardware attached, and it lets the same code be dry-run on the Raspberry Pi before the motors are wired, which takes the risk out of first power-on.

Table 10 - Hardware Control Software Modules

Module	*Responsibility*
servo_config.py	Single source of calibration truth joint→channel map, per-channel center_deg / sign / gain, joint limits, gripper endpoints, pulse-width settings.
arm_driver.py (ArmDriver)	Hardware abstraction. Converts joint radians to per-channel servo degrees, applies the mirror and limit logic, writes to the PCA9685; provides smooth (rate-limited) homing and a release (limp) function.
arm_keyboard_teleop.py	Operator interface. Reads keystrokes, maintains joint targets, and calls ArmDriver.
servo_calibrate.py	Interactive single-channel tool for finding each servo's centre, travel range, and gripper endpoints.

The teleoperation loop uses non-blocking, single-keystroke (cbreak) terminal input, so the operator can jog joints one nudge at a time; each keypress moves a joint target by a set step (0.10 rad by default) and issues the command at once. The keymap matches the simulation's teleop exactly, which preserves the operator's muscle memory across phases (Table 5.4). For larger automated moves, such as driving to the home pose at start-up or resetting, the driver interpolates all joints to the target at a bounded angular rate (about 50 Hz update, with a

configurable slew limit), so the arm eases into position rather than snapping and the structure and gears are spared sudden loads.

Table 11 Teleoperation Key bindings

Key(s)	*Action*
a / z	j1 + / − (base yaw)
s / x	j2 + / − (shoulder, both servos)
d / c	j3 + / − (elbow)
f / v	j4 + / − (wrist pitch)
g / b	j5 + / − (wrist roll)
o / p	gripper open / close
space	re-send (hold) current pose
h	smooth move to home pose
[/]	decrease / increase step size
k	release servos (go limp — arm must be supported)
q	quit (servos keep holding)

In simulation, joint commands go out as trajectory_msgs/JointTrajectory messages to two ros2_control JointTrajectoryControllers arm_controller for j1 to j5 and gripper_controller for the gripper. These run inside Gazebo through the gz_ros2_control plugin, which applies the URDF effort and velocity limits and enforces the gripper's mechanical coupling. The simulation sits on a current, supported toolchain ROS 2 Jazzy Jalisco on Ubuntu 24.04, the Gazebo physics simulator (gz sim 8, Harmonic) via ros_gz, ros2_control with gz_ros2_control, and MoveIt 2 for planning [18]. Gazebo Classic was avoided on purpose, since it is end-of-life on Jazzy. The project is split into three ROS packages rotatedauto (robot description, Gazebo bring-up, keyboard teleop), rotatedauto_moveit_config (MoveIt configuration), and emg_control (sEMG inference and the gesture-to-arm bridge).

A MoveIt 2 configuration was generated and wired into the running Gazebo controllers, adding collision-aware planning on top of the raw joint control. Two planning groups are defined, arm (the chain base_link → l5, i.e. j1 to j5) and gripper (j6l), each with named states for quick commanding (arm home and ready; gripper open and closed). The self-collision matrix was generated headlessly, disabling 21 link pairs that are either adjacent or never collide. A

moveit_simple_controller_manager maps the arm and gripper groups onto the same FollowJointTrajectory controllers the teleop uses, so planned trajectories run through the same command contract as every other motion source. As noted earlier, the KDL inverse-kinematics solver assumes a 6-DOF wrist; with only five arm DOF, joint-space and named-state goals are reliable while full Cartesian goals are best-effort.

The sEMG control mode is where the idea that the keyboard is just one source of joint commands becomes concrete. It lives in the emg_control package as two ROS 2 nodes. The emg_inference_node takes in the 2-channel sEMG stream, over USB serial or the Wi-Fi/TCP relay, treated identically, holds a 200 ms sliding window with a 100 ms hop, extracts features (MFCC plus time-domain descriptors), classifies the current gesture, smooths the result by majority vote over the last five predictions with a REST confidence gate, and publishes a stable gesture label along with the full classification result. The emg_bridge_node turns gestures into motion using the same incremental-target logic as the keyboard teleop EXTENSION and FLEXION jog a selected joint in the + and − directions, GRIP toggles the gripper (with a cooldown so signal chatter cannot double-toggle), and REST, or any absence of a confident gesture, or a stale signal, commands HOLD through a deadman timeout. It publishes to the same arm_controller and gripper_controller topics as the keyboard teleop, so the actuation layer needs no change. Because the classifier is not accurate yet (it is waiting on a re-collected, higher-quality sensor dataset), the bridge deliberately stacks three independent safeguards so a misclassification cannot cause dangerous motion majority voting on the inference side, a confidence threshold on the bridge side, and the deadman timeout that returns the arm to HOLD whenever the signal is uncertain or interrupted.

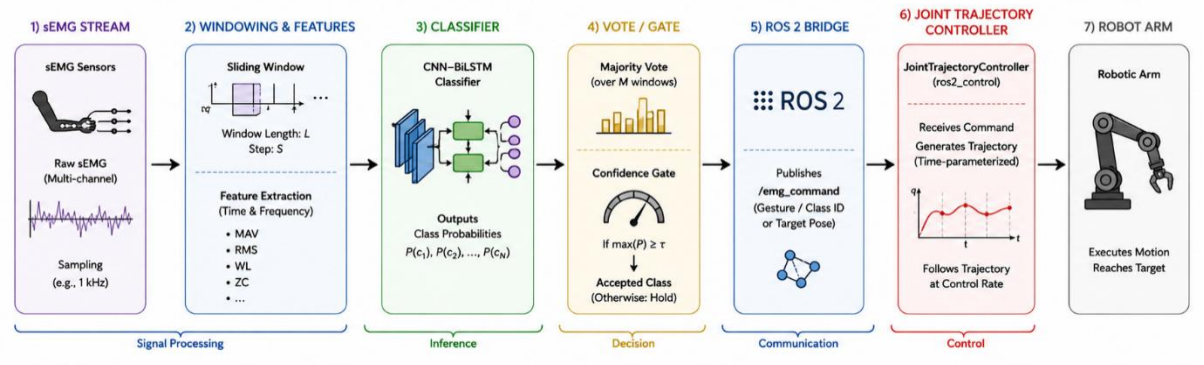


Figure 15 sEMG data flow

For the portable, real-arm setup, the control system is spread across three devices on one 2.4 GHz network the ESP32-S3 for sEMG acquisition (Chapter 4), the Raspberry Pi as a relay, and

the control host that runs ROS 2 inference, the bridge, and the simulated or physical arm (See figure 15). The ESP32-S3 serves the 2-channel sample stream from a TCP server on port 5005 (advertised over mDNS as esp32.local), batching roughly 10 ms of samples per TCP write so that network I/O never disturbs the 500 µs sampling schedule. The Raspberry Pi runs a small TCP client (tcp_receiver.py) that relays the byte stream into a pseudo-terminal, so the existing serial-based ROS consumer reads it unchanged, which makes the relay a zero-modification integration point. Since the wire format matches the direct-USB stream exactly, the same inference pipeline serves both the bench (USB) and the deployed (Wi-Fi) cases.

The single biggest practical constraint is current. MG995 servos pull on the order of an ampere each under load, several at once go past 5 A, and stall transients are higher still. So the control electronics and the motor power are kept apart the PCA9685's logic supply comes from the Raspberry Pi's 3.3 V rail and the I²C lines go to the Pi, while the PCA9685's V+ servo rail is fed from a dedicated 5–6 V supply rated for at least 5 A. The servos are never powered from the Pi's 5 V pin, and all grounds are tied together so the PWM and I²C signals share a common reference. Software safety behaviours back up the electrical design joint limits stop servos being commanded past their travel, which avoids sustained stall and overheating; on exit the servos hold their last pose rather than going limp, which is safer for a loaded arm, and relaxing them takes a deliberate operator action; and the start-up homing sequence includes a countdown so the operator can support the arm by hand on first power-on.

4.3.1 Simulation and Validation

The control software was validated step by step, much of it heedlessly. It is worth separating what has been verified in simulation and software testing from what is designed and ready but still waiting on physical hardware. Verified the URDF parses and check_urdf reports the expected kinematic tree; Gazebo spawns the arm, all three controllers (joint_state_broadcaster, arm_controller, gripper_controller) reach the active state, joint_states publishes, and the arm rests stably upright under gravity; a MoveIt 2 joint-space plan was planned and run in Gazebo (for instance j2 → 0.6 rad, j3 → −0.6 rad), reaching and holding the commanded configuration; the keyboard teleop and the sEMG bridge jogged the right joints and toggled the gripper correctly, including the inverted-sign convention and the passive-finger mimic; a recorded sEMG CSV was replayed through the inference node into the bridge, driving the simulated arm and exercising the whole chain (stream → features → classifier → gesture → joint command) without the live sensor; and the hardware control logic was run through the mock servo back end and --dry-run, taking the risk out of first power-on with no motors connected.

The strongest single piece of evidence is the shoulder hold–sag test, which both checks that the simulation enforces actuator limits and confirms the dual-servo decision. With the shoulder effort capped at a single MG995's 1.47 N·m, j2 could not hold the arm horizontal; it needs roughly 2–2.5 N·m and sagged. With the dual-servo effort of 2.94 N·m, the joint holds commanded to 0.785 rad, it settled at 0.785 rad with essentially no residual velocity. That confirms both that the doubled torque budget is genuinely needed and that gz_ros2_control's position interface enforces the URDF effort limit faithfully. Together with the successful MoveIt 2 plans and with both the keyboard teleop and the playback-driven sEMG bridge running through the same controller interface, this validates the shared-command-contract architecture the same joint-command seam is used by a human, a planner, and the sEMG pipeline without modification, which is the central design claim of the chapter.

Two parts are implemented and ready but not yet validated on the physical system, and the chapter does not pretend otherwise. Motion on the physical arm has been verified in logic, through the mock back end and dry-run, but has not yet driven the real servos under load, which is waiting on an adequately rated power supply. sEMG control has been validated end to end through recorded-signal playback, but live closed-loop control with a trustworthy classifier is waiting on the re-collected, higher-quality dataset described earlier. The present sEMG control is best understood as a validated pipeline waiting for a better dataset, not a finished classifier.

4.3.2 Conclusion and Future Work

This chapter set out the design, implementation, and validation of the control system that turns motion commands into coordinated movement of the 6-DOF arm. The architecture spans two phases under one shared operator contract a ROS 2 / Gazebo simulation path that publishes JointTrajectory commands through ros2_control, and a standalone, ROS-free hardware path that maps joint angles in radians, through per-joint calibration, into PCA9685-generated servo pulses. The arm was modelled kinematically, as an eight-link tree with five actuated arm joints, a mirrored dual-servo shoulder, and a mimic-coupled gripper, and at the actuator level, with MG995 stall torque and no-load speed setting the URDF effort and velocity limits. The deployed control was shown to be open-loop, and the modelling was framed to match, without reaching for feedback-control tools the system does not implement. In simulation the design was validated across the full stack, including the shoulder hold–sag test that confirmed the dual-servo shoulder is necessary, MoveIt 2 planning, keyboard teleoperation, and an end-to-end sEMG pipeline driven by recorded-signal playback.

The main limitations follow directly from the design choices. First, the system is open-loop with no position feedback from the MG995 servos, it cannot reject disturbances, correct steady-state error, or recover a joint from a pose where gravitational torque briefly exceeds servo torque. Second, the arm's 5-DOF kinematics, against a KDL solver that assumes a 6-DOF wrist, restrict reliable inverse kinematics to joint-space and named-state goals rather than arbitrary Cartesian poses. Third, the system is sensitive to power-supply quality an under-rated source sags under the combined servo load, the rail voltage collapses, the servos lose torque, and the PCA9685 can reset and wedge the I²C bus, a brownout failure whose symptom is servos that hold but barely move and then appear to freeze. The mitigations are a properly sized supply (≥ 5 A at 5–6 V) and software that reports I²C write failures explicitly and can relax idle channels, turning a silent hang into a clear, recoverable fault. Fourth, the sEMG control mode, though architecturally complete and validated by playback, is currently bounded by the classifier's accuracy rather than by the control layer.

The biggest opportunity for future work is moving from open-loop to closed-loop control. Adding joint-position feedback, for instance servos or encoders that report the achieved angle, would allow disturbance rejection, steady-state error correction, and recovery from extreme poses, and it would make the classical modelling and stability tools (transfer functions, root locus, Bode analysis) genuinely applicable. A 5-DOF-aware or position-only inverse-kinematics solver would lift the Cartesian-goal limitation. On the hardware side, finishing the physical bring-up with the right power supply is the immediate next step, followed by validating the hardware control path under load against the simulation predictions. Most importantly, the control layer is built so the keyboard is just one source of joint commands the same ArmDriver interface is the integration point for the sEMG-driven mode, so the classifier's outputs map onto the identical joint-command API, swapping decoded muscle intent in for the operator's keystrokes without touching the actuation layer underneath. That continuity, from the simulation interface to the keyboard hardware interface to the future sEMG interface, is the central theme of the control system, and it is what lets the larger goal, direct real-time sEMG control of a physical arm, be reached step by step and safely.

4.4 EMG Acquisition

The acquisition front end streams two envelope channels at 500 Hz from the ESP32-S3. Channel 1 is a Muscle Sensor v3 (a hardware-rectified envelope, sampled on GPIO3 through a 56 kΩ / 22 kΩ divider) on the forearm flexor group; channel 2 is a CHEEZ.semg module (an on-device bandpassed envelope on GPIO7, with a digital electrode-contact line on GPIO6) on the extensor group. This two-sensor setup replaced an earlier plan of two identical Muscle Sensor v3 boards sampled at 2000 Hz a single cleaner on-device envelope per muscle group was the better trade, and the rate was dropped to the CHEEZ module's native 500 Hz. Both raw ADC streams are logged for offline processing rather than filtered on the device.

4.4.1 Collection Software and Recording Protocol

Recordings were run from a custom collection application (collection_gui.py) that lines the visual cue up with the ESP32 firmware. For each trial it shows the target gesture as a name, a short instruction, and a reference photo; runs a fixed four-second rest countdown, during which the participant settles back into a relaxed neutral posture; then sends a trigger byte to the ESP32 and opens a timed recording window, shown as a live progress bar, before moving on to the next gesture automatically. It does the participant-ID handshake with the board and writes each session to a timestamped CSV next to a metadata file that records the per-channel sensor type and pin. Running this way, automated and self-paced, keeps the cue timing, the gesture order, and the trial boundaries identical across every cycle and every participant.

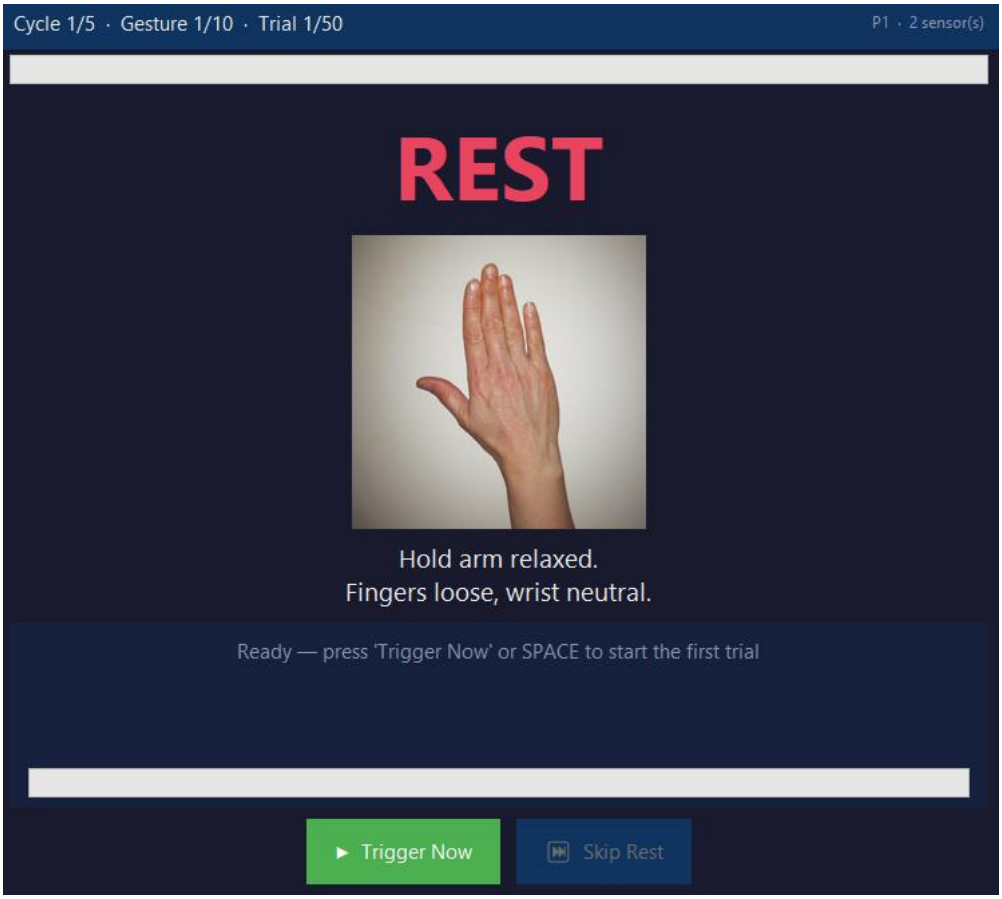


Figure 16 Data-collection application showing the gesture cue

Each session follows a fixed protocol the full gesture set is repeated over five cycles, in the same order each cycle, with the cued four-second rest between consecutive gestures. In the session used for all the modelling here (recording P1_20260606_231716), twelve gestures were recorded over five cycles, with five seconds, 2 500 samples at 500 Hz, captured per gesture per cycle. The twelve were rest, extension, flexion, ulnar deviation, radial deviation, grip, two directions of downward forearm rotation, two directions of frontal forearm rotation, and elbow extension and retraction.

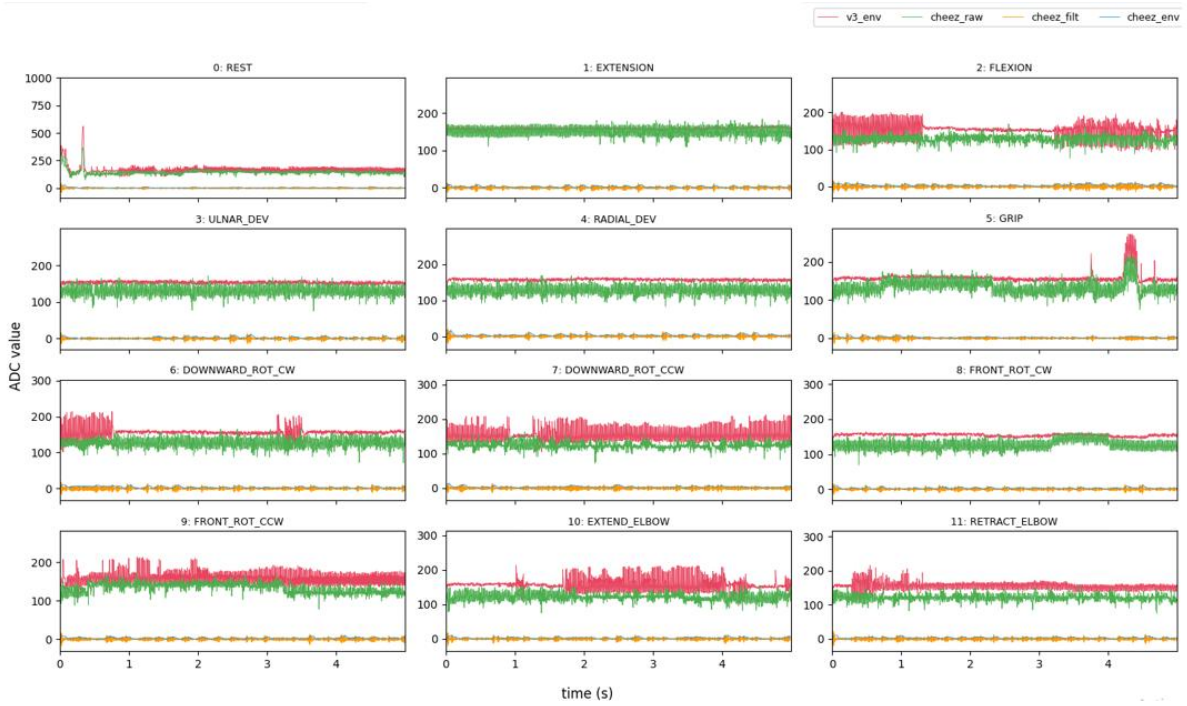


Figure 17 - Raw gesture signals

4.4.2 Electrode Placement

The two sensors went over specific forearm muscles, following SENIAM surface-electrode guidelines. The Muscle Sensor v3 sat over the flexor carpi radialis on the anterior forearm and the CHEEZ.semg over the extensor digitorum on the posterior forearm, each with its differential electrode pair lined up along the muscle belly and a reference electrode on a nearby bony landmark for common-mode rejection. These two muscles were picked as accessible, well-separated stand-ins for the wrist-flexor and finger-extensor compartments whose activity tells the target gestures apart. The skin was cleaned before the electrodes went on to lower contact impedance. Figure 18 shows where each sensor was placed.



Figure 18 sEMG sensor placement on the forearm flexor and extensor muscle groups

4.4.3 Data-Collection Campaign and Data Validity

A data-collection campaign was run with thirty-six participants, aiming for a multi-subject dataset. In practice almost none of those recordings turned out usable. Two failure modes dominated. The wireless link dropped and corrupted samples mid-stream, and electrode contact was inconsistent skin preparation was sometimes too light, electrodes lifted during movement, and the CHEEZ channel was not working. Together these left sessions with missing or unreliable signal. After quality screening, only the author's own recordings were clean enough to keep, which is why the modelling in this chapter rests on a single participant. This is a hardware- and procedure-level data-quality failure, not a modelling choice, and it is the root cause of the single-subject, cross-cycle generalisation limit discussed in Chapters 5 and 6. The obvious remedies, wired or buffered transmission to stop sample loss, careful skin preparation, and per-trial electrode-contact checks, follow straight from these observations and are taken up as future work.

4.4.4 Analysis and Simulation Pipeline

Two software toolchains sit behind the analytical work. Signal processing, feature extraction, and model training were done in Python (NumPy, SciPy, scikit-learn, librosa, and PyTorch).

The robotic arm's system-level behaviour was validated in a ROS 2 Jazzy / Gazebo Harmonic (gz sim 8) physics simulation with MoveIt 2, driven from the mass-accurate URDF exported out of the SolidWorks assembly. The simulation is a risk-free place to exercise the kinematics, joint limits, and gesture-to-joint mapping before any command reaches the physical hardware, and it is the main way the arm's response to classifier output is predicted.

4.4.5 Feature Extraction and Model Pipeline

Classification works on short windows rather than whole trials, so the same computation can run live. Each 200 ms window (100 samples) advances by a 100 ms hop (50 samples) and is AC-coupled by per-channel median subtraction, the only normalisation a live system can reproduce from a single window, and one that keeps the amplitude information that carries most of the discriminative power. From each window a 76-dimensional feature vector is built, laid out as in Table 6.1 per channel, statistics of eight MFCCs (mean, standard deviation, maximum, minimum) plus six time-domain descriptors, so $(8 \times 4 + 6) \times 2 = 76$ features.

Table 12 - Feature extraction

Stage	***Setting***
Window length	200 ms (100 samples at 500 Hz)
Hop	100 ms (50 samples), 50 % overlap
Per-window normalisation	AC-couple (per-channel median subtraction)
MFCC features	8 MFCCs × {mean, std, max, min} per channel
Time-domain descriptors	RMS², sparseness, irregularity, RMS, waveform length, zero crossings
Feature vector	(8×4 + 6) × 2 channels = 76 features
Windows per class	240

Two model families were trained on these windows. The classical family is a support vector machine, a random forest, and a gradient-boosting classifier, each inside a StandardScaler pipeline. The deep family is a compact recurrent convolutional network (CRNN) that works on the raw 100 × 2 window rather than the summary features three 1-D convolutional blocks (kernel 5, batch-norm, ReLU, max-pool) feed a bidirectional GRU whose outputs are mean-pooled over time and passed to a fully connected classifier, 99 500 parameters in all, trained

with class-weighted cross-entropy, label smoothing, and a cosine-annealed AdamW schedule. A deeper 2-D spectrogram CNN (620 k parameters) was also tried as a baseline.

To keep the live system identical to training, one model is loaded and queried on each window. The torch CRNN is wrapped so it presents the same predict / predict-probability interface as the scikit-learn models and returns a full, gesture-id-indexed probability vector; the wrapper serialises through joblib (state dictionary only), so the model moves cleanly between machines and library versions. Keeping the original gesture identifiers, instead of re-indexing, means dropping classes leaves the whole preprocess → train → inference → ROS pipeline index-compatible with no code change, and keeps rest gated at index 0.

The physical platform is a custom 6-DOF arm designed in SolidWorks and printed in ABS by fused-deposition 3-D printing, weighing 2.41 kg, standing 670.24 mm tall on a Ø150 mm base. It has eight links and six joints, detailed in Chapter 3. Six MG995 metal-gear servos drive it through a PCA9685 16-channel I²C PWM driver (address 0x40, 50 Hz), with the shoulder j2 driven by two mirrored servos to share the cantilevered load. The base turntable j1 runs on a stepper motor through a DRV8825 step/direction driver, a late change from the original base servo, made to get smooth, repeatable continuous rotation of the turntable; the stepper is a bipolar unit, most likely a NEMA 17, pending bench confirmation. The servos and the stepper run off a dedicated 5–6 V supply with a common ground to the Pi and driver, never off the Pi rail.

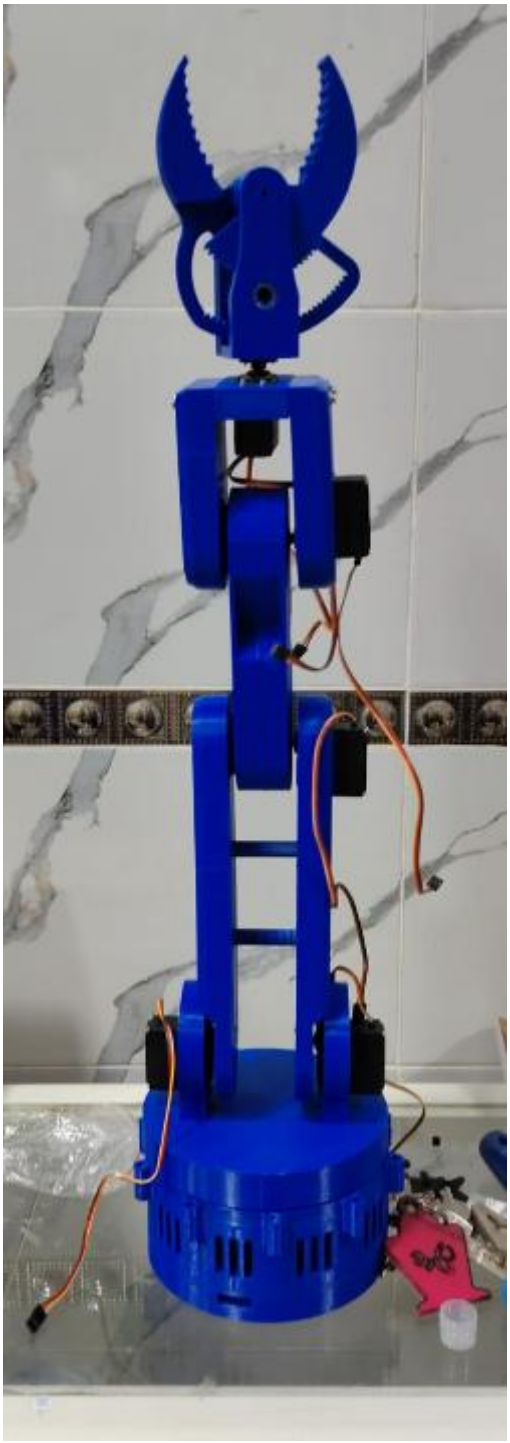


Figure 19 The fabricated 6-DOF arm and gripper

4.4.6 End-to-End Test Configuration

The full gesture-control loop was put together and tested wirelessly. Updated firmware streams the 500 Hz two-channel envelope over Wi-Fi/TCP (the ESP32 acting as its own access point at 192.168.4.1, or as a station on an existing 2.4 GHz network) and mirrors the same data to USB, so a PC running the Gazebo simulation and the Raspberry Pi driving the physical arm can both take the live stream. On the receiver, the real-time inference program reads the stream,

windows it, extracts features, and classifies each window, applying a majority vote and a rest gate before it issues motion commands.

4.4.7 Mains-Interference Experiment

During live testing the Muscle Sensor v3 envelope turned out to carry a strong, roughly stationary 50 Hz pedestal (Egyptian mains) and its harmonics, which the sensor's hardware envelope does not remove. A causal streaming notch filter, an IIR notch with persistent per-channel state seeded to the first sample to avoid start-up ringing, was put in at 50 Hz and every harmonic below Nyquist, applied to each incoming sample before windowing. Two things came out of this. First, the notch shifts the 76-dimensional feature vector by roughly 71 % in relative terms; it is not absorbed by window normalisation, so a model trained without the notch but run with a live notch predicts almost at random, which means training and inference have to use a matching notch. Second, since a real-time stream cannot reproduce the zero-phase filtering normally used offline, training was switched to the same causal notch, guaranteeing that the training and inference signal paths match.

4.4.8 Evaluation Methodology and a Corrected Leakage Pitfall

Because the 200 ms windows overlap by 50 %, a naïve stratified cross-validation drops a window and its half-overlapping neighbour into different folds, which lets the model partly memorise trials. That optimistic protocol gave a random-forest accuracy of about 57.7 %, but it measures within-session separability, not generalisation. Every headline figure in this chapter instead uses a leakage-free, deployment-realistic split that holds out whole recording cycles (train on cycles 1–3, validate on cycle 4, test on cycle 5), with a leave-one-cycle-out check used during gesture selection. Each accuracy below is labelled with the protocol that produced it.

4.4.9 Gesture-Subset Selection

Under the leakage-free cross-cycle protocol the twelve-class problem transfers poorly only rest and a few gestures stay reliably above chance, the signature of cross-cycle drift on a single participant. So a control-oriented subset of seven classes was chosen, rest plus three antagonistic pairs (wrist extension/flexion, two forearm-rotation directions, and elbow extension/retraction), keeping the original gesture identifiers so the pipeline stays index-compatible. Ulnar and radial deviation, grip, and the frontal-rotation pair were dropped as near-chance under this protocol.

4.5 Control System

4.5.1 Introduction

A control system turns high-level motion intent into coordinated movement of an actuated mechanism. Here it is the last stage of the sEMG-to-motion pipeline it takes discrete motion commands, from an operator at a keyboard or, in the end, from the sEMG gesture classifier, and turns them into joint angles for the arm's actuators, which are six MG995 servos and a stepper-driven base turntable. This is the layer where the classifier's abstract output becomes real motion of the 6-DOF arm.

The control architecture is built to span the project's two delivery phases. In the first, the arm is commanded inside a ROS 2 and Gazebo simulation, which is used to check the kinematics, joint limits, and operator interface before risking the physical hardware. In the second, the same conventions drive the actuators of the real arm straight from a Raspberry Pi. Both paths present the operator with an identical contract the same commands move the same joints in the same directions, with the same limits and sign conventions. So, behaviour validated in simulation carries straight over to hardware, and the future sEMG control mode can drop in at a single, well-defined point.

Before designing a controller, the plant, meaning the arm and its actuators, must be modelled. It is worth being clear from the start that the implemented controller is open-loop position control a commanded joint angle becomes a servo pulse width, and the servo's own electronics drive the horn to that position. There is no external feedback of the achieved angle, because the MG995 servos expose no position signal to the host. That rules out the classical closed-loop analysis tools, open-loop transfer functions, root-locus plots, and Bode frequency response, none of which apply to the deployed system, so none are presented here. Each MG995 does run its own internal position loop, through an integrated potentiometer and analogue control IC, but that loop is sealed inside the servo and the project's software cannot reach it. The model built here is therefore kinematic and actuator-capacity based it predicts where the arm will go and whether the servos can hold a given pose, not the dynamic response of a feedback loop.

4.5.2 System Modelling and Analysis

The arm is a 6-DOF design with five serial arm joints plus a gripper, exported from SolidWorks as a URDF model. Anchored to the world frame, the kinematic tree runs world → base_footprint → base_link → l1 → l2 → l3 → l4 → l5 → {l6l, l6r}. The first two joints are fixed and only serve to anchor and orient the arm upright on the floor. The five actuated arm

joints are j1 to j5, and the gripper comes from the l5→l6l (driver) and l5→l6r (passive follower) pair. The URDF, generated from a xacro macro, is the single source of truth for geometry, kinematics, and limits, so the simulation, the motion planner, and the hardware driver all work from one description. Table 5.1 lists the actuated joints.

Table 13 Actuated Joints of the 6-DOF Arm

Joint	*DOF role*	*Servos*	*Limit (rad)*	*PCA9685 ch*
j1	Base yaw	1	continuous	0
j2	Shoulder pitch	2 (mirrored)	±1.5708 (±90°)	1, 2
j3	Elbow pitch	1	±1.5708 (±90°)	3
j4	Wrist pitch	1	±1.5708 (±90°)	4
j5	Wrist roll	1	continuous	5
j6	Gripper	1 driven + 1 passive	−0.51 → 0.288 (open → closed)	6

Two modelling details deserve emphasis, because both shape the control design. The first is the dual-servo shoulder (j2). The arm has six controllable degrees of freedom but seven actuators, six MG995 servos and a stepper-driven base. The shoulder carries the heaviest gravitational load, the cantilevered mass of the extended arm, so it is driven by a pair of MG995 servos working together. They sit on opposite sides of the shoulder axis and so must turn in opposite directions to make one motion; they are mirrored. That is why the control software treats j2 specially, as described below. The second is the mimic gripper (j6). One servo drives the two-finger gripper, with the opposite finger following through a passive linkage. In the URDF this is a driver joint (j6l) and a passive follower (j6r) coupled by a mimic tag with multiplier −2.0885, which captures the physical 1.13 2.36 reverse gear ratio, the negative sign encoding the opposing motion of the fingers. By convention +0.288 rad is closed and −0.51 rad is open, and that inverted convention carries through to the teleoperation and the future sEMG mapping.

Table 14 Servo control specifications

Parameter	*Value*
Servo model	MG995 metal-gear servo
Stall torque (per servo, 6 V)	1.47 N·m

No-load speed (per servo, 6 V)	8.05 rad/s
Shoulder (j2) combined torque	2.94 N·m (two mirrored servos)
Servo control signal	PWM, 50 Hz, ≈500–2500 µs (≈0°–180°)
PWM driver	PCA9685, 16-channel, 12-bit, I²C 0x40
Teleoperation step size	0.10 rad per keypress (configurable)
Gripper convention	+0.288 rad = closed, −0.51 rad = open
Gripper mimic ratio	−2.0885 (1.13 2.36 reverse, opposing)
Servo power rail	5–6 V, ≥ 5 A dedicated supply

The arm's dynamic capability is set by its actuators. Joints j2 to j5 and the gripper each run on one or more MG995 metal-gear servos; the base j1 runs on a stepper motor through a DRV8825 module but, like every other joint, is modelled in simulation as a position-controlled joint. At 6 V the relevant MG995 figures are a stall torque of about 1.47 N·m and a no-load speed of about 8.05 rad/s. These populate the URDF's per-joint effort and velocity limits, so the simulated rigid-body dynamics reflect the real actuator capacity the single-servo joints (j3, j4, j5, gripper) are capped at 1.47 N·m, while the dual-servo shoulder j2 is modelled at 2.94 N·m (2 × 1.47). The doubled torque at j2 is a requirement, not a luxury as the validation below shows, a single MG995 cannot hold the extended arm horizontal against gravity, where the holding torque needed is around 2–2.5 N·m, and the joint sags. Table 5.2 brings together the actuator and control parameters.

Because a real servo cannot spin endlessly and the links can collide, every joint is bounded by software limits in radians that mirror the URDF limits (Table 5.1). Commands are clamped against these before they go out, which keeps a servo from being driven into a mechanical hard stop, where an MG995 would stall and overheat. Two modelling limits bound what the model can say. First, being open-loop with no position feedback, it cannot represent steady-state position error, disturbance rejection, or a joint recovering from a pose where gravitational torque briefly exceeds available servo torque; in simulation a joint may hold a partly loaded pose yet be unable to lift back out of it, which is exactly what an under-powered, feedback-less actuator would do. Second, the arm has five actuated arm joints, while a general position-and-orientation inverse-kinematics solution needs six. The KDL solver in the motion-planning layer assumes a 6-DOF wrist, so arbitrary Cartesian goals often have no solution. That is inherent to a 5-DOF arm, not a software bug joint-space and named-state goals are reliable, while full Cartesian goals are best-effort.

4.6 Prototyping and Testing

4.6.1 Motion-Control Testing

A noisy classifier drove the arm erratically at first, with rapid gesture flips piling up into simultaneous multi-joint motion. The motion bridge was therefore rebuilt around discrete, debounced commands every three seconds it takes the majority gesture over the interval and, as long as the winner is not rest and clears a minimum vote count, fires one fixed-size move on a single joint while freezing every other joint at its current measured position. The result is exactly one deliberate joint motion per decision, a conscious trade of responsiveness for stability that suits a noisy classifier and matches the physical arm, where each servo holds its last set-point.

This section reports the measured classification results on the seven-class control set, the per-class behaviour of the deployed model, the feature-separability analysis, and the simulation validation. Chance accuracy on seven classes is 14.3 %.

4.7 Results

4.7.1 Classification Accuracy

Table 6.2 compares the model families with the mains notch off and on, under the cross-cycle test split. The notch is a real, repeatable improvement rather than just hygiene it took the best classical model from 25.0 % to 38.4 % test accuracy (about 2.7 × chance) and lifted the CRNN's best validation accuracy from roughly 15 % to about 35 %. The random forest with the causal notch is the deployed model it gives the highest cross-cycle test accuracy, is lighter, and behaves more smoothly than the deep models.

Model	***Mains 50 Hz***	***Val acc.***	***Test acc.***
Classical (gradient boosting)	off	16.1 %	25.0 %
Classical (random forest) — deployed	on (causal)	30.7 %	38.4 %
Recurrent CNN (99.5 k params)	off	14.9 %	31.8 %
Recurrent CNN (99.5 k params)	on (causal)	34.8 %	32.7 %
2-D spectrogram CNN (620 k params)	on (causal)	28.6 %	28.6 %

The CRNN reaches slightly higher validation accuracy with the notch, but the random forest is preferred on the deployment-realistic test split, and the CRNN's training accuracy climbing to 100 % while its validation accuracy stays low is a clear sign of over-fitting on single-participant, drift-heavy, effectively single-channel data. For context, the leaky mixed-cycle protocol gave about 57.7 %, evidence that the underlying signal does exist within a session, while a leave-one-cycle-out check fell to roughly 12–18 %, which puts a number on the drift that bounds real-world use.

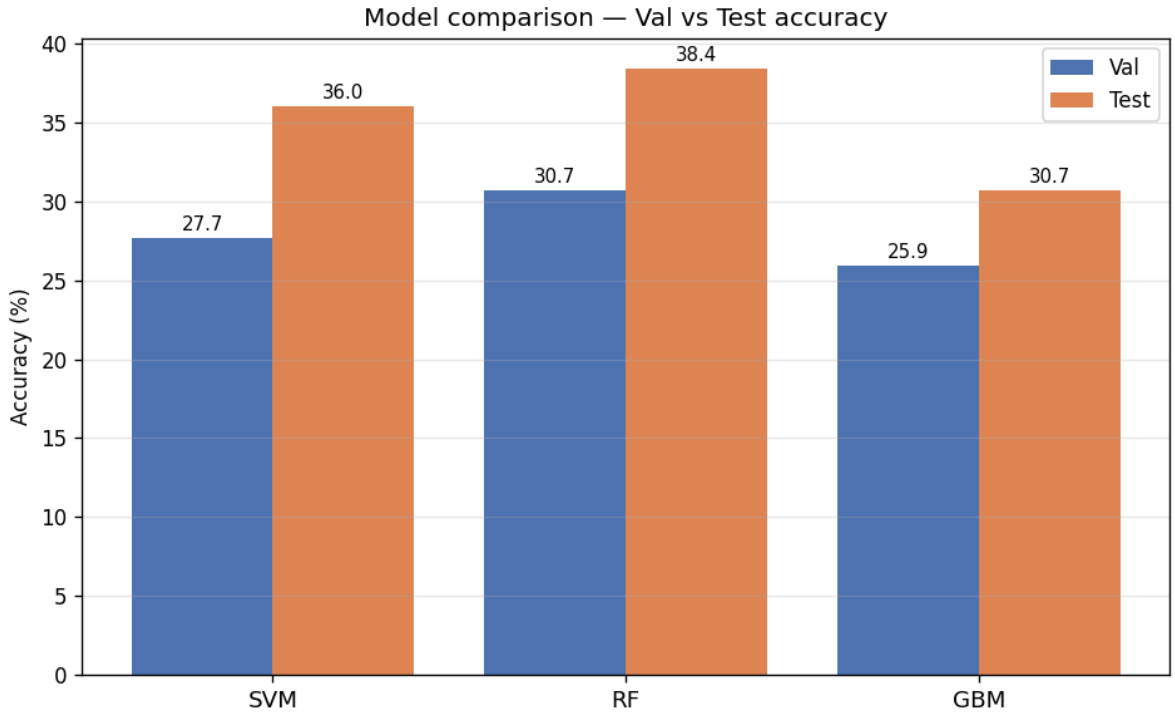


Figure 20 Model comparison

4.7.2 Per-Class Performance of the Deployed Model

The aggregate accuracy hides a very uneven per-class picture. As the confusion matrix (Figure 21) and Table 6.3 show, the deployed random forest is carried almost entirely by two classes rest is recognised in 47 of 48 test windows (F1 = 0.92) and elbow extension in 45 of 48 (F1 = 0.53). Elbow retraction collapses into elbow extension and is never identified correctly, while wrist extension, flexion, and the downward-rotation pair are mutually confused; ulnar

deviation, radial deviation, grip, and the frontal-rotation classes are effectively never predicted. The macro-averaged F1 across the seven classes is 0.29.

Class	***Precision***	***Recall***	***F1***
Rest	0.87	0.98	0.92
Extension	0.21	0.38	0.27
Flexion	0.00	0.00	0.00
Downward rotation (CW)	0.26	0.40	0.31
Downward rotation (CCW)	0.00	0.00	0.00
Extend elbow	0.37	0.94	0.53
Retract elbow	0.00	0.00	0.00
Macro average	—	—	0.29

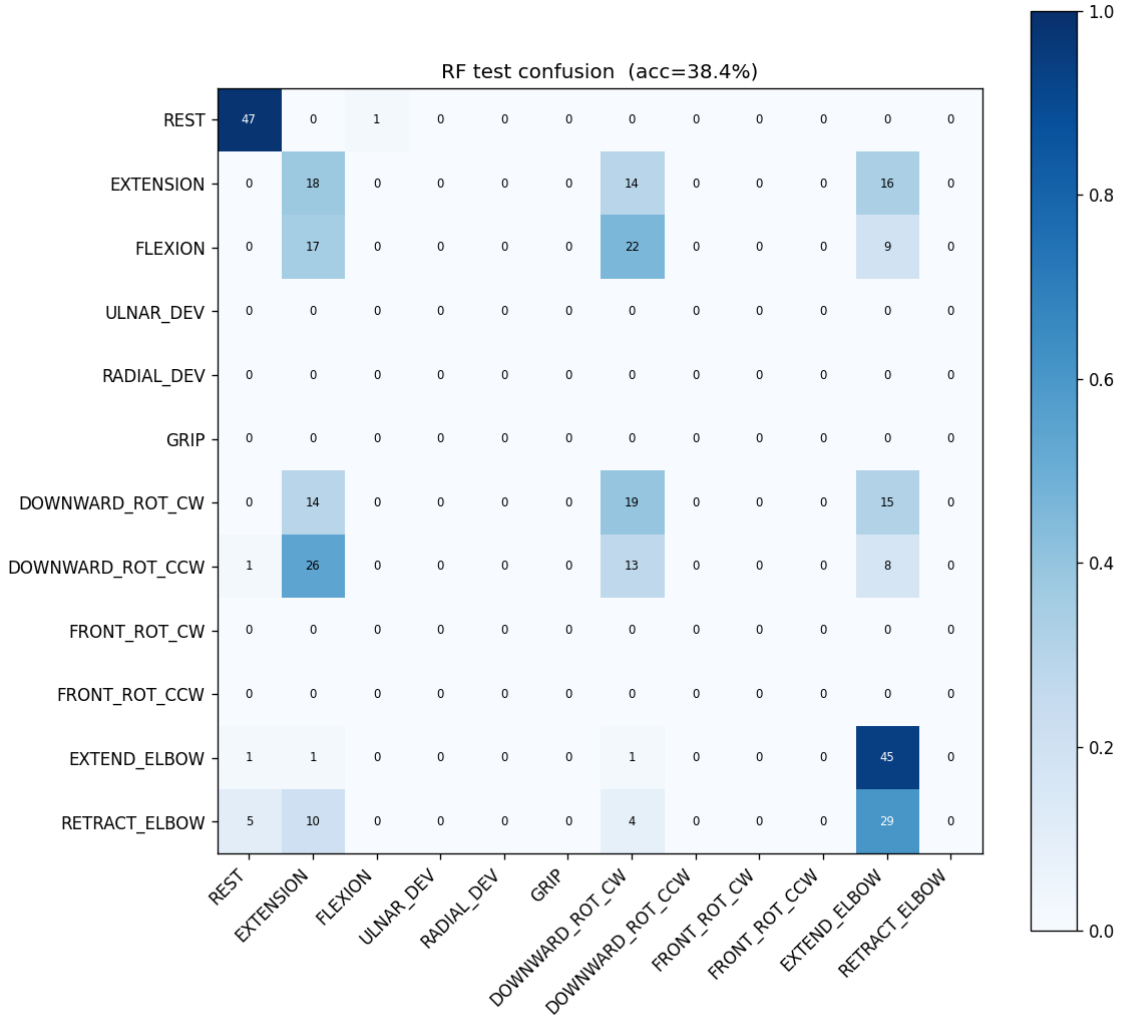


Figure 21 Confusion matrix of the deployed random forest on the held-out test cycle

4.7.3 Feature Separability

A one-way ANOVA F-test ranking of the 76 features (Figure 6.5) confirms both where the discriminative information sits and why the system is effectively single-channel. The most separable features all come from the Muscle Sensor v3 channel, waveform length, sparseness, the low-order MFCC statistics, and RMS, and they separate rest and the high-amplitude elbow movements cleanly while overlapping heavily for the low-amplitude wrist and rotation gestures. The CHEEZ-channel features rank far lower and show little class structure, in line with that channel having failed to couple to the ADC during recording.

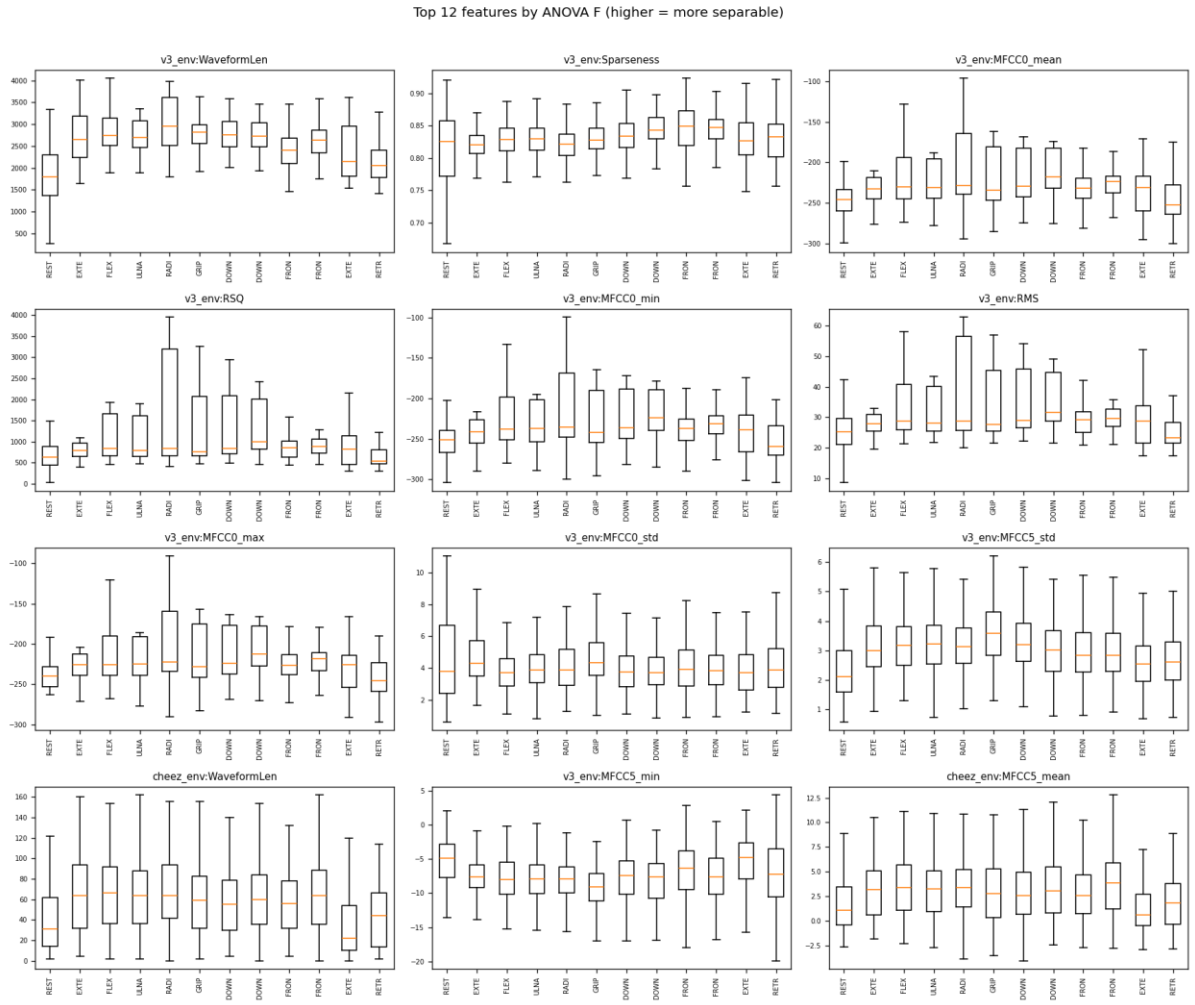


Figure 22 Top features ranked by ANOVA F-value (class separability)

4.7.4 Simulation and System Validation

The robotic side was validated mainly in simulation and through software dry runs. The URDF parses and brings up cleanly in Gazebo; MoveIt 2 plans collision-aware joint-space motions for the arm; and the teleoperation and sEMG-bridge nodes drive the simulated joints correctly, including from recorded EMG replayed through the live pipeline. The strongest single piece of physical evidence is the shoulder hold–sag test, which confirms both that the simulation enforces the actuator torque limits and that the dual-servo shoulder is necessary a single MG995 cannot hold the extended arm horizontal, whereas the mirrored pair can. Two parts are implemented but not yet validated on hardware, sustained motion of the physical arm under load (waiting on a suitably rated power supply) and live closed-loop EMG control with an accurate classifier, and that is stated plainly rather than claimed.

4.7.5 Summary of Findings

Taken together, the experimental work delivers a working, wireless, debounced, end-to-end sEMG-to-arm system across both simulation and hardware, one whose classification accuracy is bounded by single-subject cross-cycle drift and a one-channel hardware fault rather than by the modelling or the control engineering.

Chapter (5) Analysis and Discussion

5.1 Summary of Findings

This chapter pulls together the findings of the experimental work, reads them against the research questions and the wider literature, and lays out the limitations that frame them. It does not repeat the detailed procedures of Chapter 4.

The research set out to test whether a deep-learning classifier could map a set of sEMG gestures to real-time control of a 6-DOF arm. The original hypothesis aimed for a CNN-BiLSTM classifier above 70 % validation accuracy on a ten-class problem and over 80 % on data from thirty or more participants, integrated through a ROS 2 / MoveIt 2 control architecture. The implemented system met the integration and simulation goals but fell well short of the accuracy and participant-count targets, for reasons the results make specific. The main findings are

- On the deployment-realistic cross-cycle split, the deployed random forest reached 38.4 % test accuracy on the seven-class control set (macro-F1 = 0.29), roughly 2.7 times the 14.3 % chance level.
- The per-class picture is very uneven rest (F1 = 0.92) and elbow extension (F1 = 0.53) are reliable, while the low-amplitude wrist and forearm-rotation gestures are mostly confused and several classes are never predicted.
- A causal 50 Hz mains-notch filter, applied the same way in training and inference, took the best classical model from 25.0 % to 38.4 % test accuracy, a repeatable measured gain rather than cosmetic signal hygiene.
- A leakage-aware evaluation showed that an apparently strong 57.7 % from overlapping-window cross-validation collapses to 12–18 % under leave-one-cycle-out, which exposes cross-session drift as the main limiter.
- The compact recurrent CNN, sound as a design, over-fits at this data scale (training accuracy reaching 100 %, validation low), so the lighter classical model was deployed instead.
- The robotic subsystem was validated in ROS 2 / Gazebo simulation, URDF bring-up, MoveIt 2 planning, teleoperation, EMG-bridge replay, and the shoulder hold–sag test, giving a working wireless, debounced, end-to-end gesture-to-arm loop.

5.2 Interpretation of Results

Read together, the results pin down where the bottleneck is. The wide gap between within-session accuracy (about 57.7 % under the leaky protocol) and cross-cycle accuracy (38.4 %, dropping to 12–18 % under leave-one-cycle-out) shows the gestures are genuinely separable

when training and testing share a session, but that the feature distributions drift between recording cycles. With one participant and one usable channel, the classifier has nothing to learn what is invariant across sessions from, so the limit is the data regime, not the model or the feature set.

The per-class pattern backs this up. The classes that survive the cross-cycle split, rest and the elbow movements, are exactly the ones with the largest, most repeatable amplitude signatures on the Muscle Sensor v3 channel, as the separability analysis confirms; the ones that fail are the low-amplitude wrist and forearm rotations, whose envelopes overlap. Elbow retraction collapsing into elbow extension, and the wrist and rotation gestures collapsing into each other, is what you would expect from shared muscle synergies seen through a single amplitude envelope. The near silence of the CHEEZ channel in the separability ranking independently backs up the hardware finding that the system ran, in effect, on one channel.

These observations line up with the wider sEMG literature. Cross-session and cross-subject non-stationarity is a well-documented obstacle in EMG gesture recognition, and deep models are reported again and again to need many subjects and sessions before they generalise, which is exactly why the higher-capacity networks here over-fit while a regularised classical model carried over slightly better. Rest being the most reliably recognised class, against a literature where rest is usually the hardest, is a direct artefact of the rectified-envelope sensor rest sits near a hard zero baseline and is easy to separate by amplitude. And the finding that mains filtering is not absorbed by per-window normalisation, so it has to be matched between training and inference, is one instance of the general rule that the offline and online signal paths have to be identical for a real-time classifier to behave as trained.

5.3 Discussion

The broader significance of the work is less its absolute accuracy than the fact that it is a complete, honestly characterised pipeline from muscle to motion. It shows that low-cost, envelope-only sensors and a Raspberry-Pi-class computer can sustain a wireless, real-time gesture-to-arm loop with safe, debounced motion, and it pins down which part of such a system limits performance. For applied EMG-controlled robotics, assistive and prosthetic devices especially, the practical lesson is that data breadth and channel integrity matter more than model choice effort is better spent on more participants, per-session calibration, and reliable electrodes than on deeper networks at this scale.

Two engineering pieces are reusable beyond this project. The causal live mains-notch with a verified train-equals-inference guarantee is a transferable recipe for any envelope sensor working on mains-contaminated signals, and the inference-conformant wrapper that makes a PyTorch recurrent model a drop-in for a scikit-learn-style real-time loop is a small but genuinely useful bit of infrastructure. The leakage-aware evaluation is itself a methodological caution with overlapping windows, ordinary cross-validation materially overstates EMG classifier performance, so cycle- or session-held-out protocols should be the default when reporting generalisation.

The work also raises ethical and societal points that fit a human-controlled actuator. A misclassifying model that moves a physical arm is a safety concern, which is why the deployed bridge is deliberately cautious, majority voting, a rest dead-man gate, and one bounded joint motion per decision, trading responsiveness for predictability. As an assistive technology it would also need informed handling of biometric EMG data and validation across the range of intended users, a need the single-participant scope here only underlines. These are framed as design obligations for any future deployment, not as solved problems.

5.1 5.4 Limitations of Study

The interpretation above must be read against several explicit limitations

A single participant and a single session for the deployed models, with no inter-subject or inter-session validation; the reported cross-cycle accuracy reflects drift within one person's data.

Effectively one working EMG channel the CHEEZ.semg channel did not couple to the ADC during recording, so the achievable separability was bounded by a single envelope.

Over-fitting of the deep models at this data volume, which is why a classical model was deployed; the deep results should be read as an architecture and pipeline contribution, not a performance claim.

An actuation gap at the base joint, between the simulation, which models j1 as a uniform position-controlled joint, and the physical arm, whose DRV8825 stepper is not yet bridged into the real-arm driver; the exact stepper model also still has to be confirmed on the bench.

Open-loop position control throughout, with no joint feedback on the physical arm, so disturbances and steady-state errors cannot be corrected.

The three-second voting debounce, which steadies motion at the cost of responsiveness and is a deliberate but real limit on interactivity.

Physical-arm motion under load and live closed-loop EMG control are implemented but not yet validated, pending a suitably rated power supply and a more accurate classifier.

Chapter (6) Conclusions

6.1 Conclusions

This research designed, built, and evaluated a complete surface-EMG gesture-controlled 6-DOF robotic arm, spanning sensing, machine learning, and ROS 2 robotic control. The headline outcome is a working, wireless, end-to-end system whose behaviour is fully and honestly characterised a low-cost envelope-sensing front end and a Raspberry-Pi-class computer can drive a physical arm in real time with safe, debounced motion, and the factors that bound its accuracy have been identified and measured rather than assumed.

Against the original hypothesis, the evidence is mixed, and it is reported that way. The system-integration goals were met, ROS 2 / MoveIt 2 control, Gazebo validation, and an end-to-end gesture-to-arm loop all work, but the classification targets (over 70 % validation on ten classes, and over 80 % across thirty or more participants) were not reached on the implemented single-participant dataset, where the deployed model managed 38.4 % cross-cycle test accuracy on seven classes. The null hypothesis of no advantage from higher-capacity temporal models was, if anything, supported at this data scale the deeper networks over-fit and a regularised classical model generalised slightly better. So, the contribution rests on the integrated system and its methodology as much as on raw accuracy

A complete, reproducible sEMG-to-arm pipeline, acquisition, windowed feature extraction, classification, and a debounced gesture-to-joint bridge, running wirelessly across both simulation and hardware.

A causal real-time 50 Hz mains-notch filter, matched between training and inference, shown to raise test accuracy from 25.0 % to 38.4 %.

A leakage-aware evaluation that exposes and corrects the optimism of overlapping-window cross-validation, giving an honest cross-cycle measure of generalisation.

Identification of the real performance bottleneck, single-subject cross-session drift and a one-channel hardware fault, rather than the model or the features.

A validated robotic subsystem, including the shoulder hold–sag test that confirms the dual-servo shoulder is necessary.

The main limitations bounding these conclusions are the single-participant, effectively single-channel dataset; the over-fitting of the deep models at this scale; the open-loop control with no

joint feedback; and the base-actuation and power-supply gaps that leave full physical-arm operation pending. None of these undermine the system as built, but each shape how its results should be read.

6.2 Future Work

The findings point to a clear, prioritised programme of further work

Collect data from many participants over multiple sessions and add a short rest-and-calibration recording at the start of each session to normalise against drift, the single highest-leverage improvement, since it addresses the dominant limitation head-on.

Repair or replace the second EMG channel so the system genuinely runs on two channels, restoring the separability lost to the CHEEZ hardware fault.

Integrate the DRV8825 base stepper into the real-arm driver with step/direction control and homing, so the physical base matches the simulation's command space, and confirm the stepper motor specification on the bench.

Re-evaluate the recurrent CNN once the dataset grows, where deep temporal models should overtake the classical baseline.

Move from open-loop to closed-loop joint control by adding position feedback and implement 5-DOF-aware (position-only) inverse kinematics for the arm.

Finish physical-arm validation under load with a suitably rated power supply and close the live EMG-to-hardware loop once classifier accuracy allows.

Appendix A – Hardware specifications

Advancer Technologies sensor specs

Parameter	Minimum	Typical	Maximum
Power Supply Voltage (Vs)	±3.5 V	±5 V	±18 V
Gain Setting, Gain = 207 × (X / 1 kΩ)	0.01 Ω (0.002×)	50 kΩ (10,350×)	100 kΩ (20,700×)
Output Signal Voltage (Rectified & Smoothed)	0 V	—	+Vs
Differential Input Voltage	0 mV	2–5 mV	+Vs/Gain

ESP32 specs

Parameter	Value
MCU	Dual-core 32-bit Xtensa LX7
Wireless Communication	Wi-Fi
Flash Memory	16 MB
PSRAM	8 MB Octal-SPI PSRAM
Reserved GPIOs	GPIO33–GPIO37 (PSRAM interface)
Recommended ADC Bank	ADC1 (GPIO1–GPIO10)
Avoided ADC Bank	ADC2 (GPIO11–GPIO20) when Wi-Fi is enabled
Application in This Work	sEMG acquisition and wireless data transmission

Raspberry Pi 4 Model B specifications

Parameter	Specification
Processor	Quad-core ARM Cortex-A72 (64-bit)
Clock Frequency	1.8 GHz
RAM	Up to 8 GB LPDDR4
Operating System	Ubuntu 24.04 LTS
Software Frameworks	ROS2 Humble, MoveIt2
Application in This Work	Real-time sEMG acquisition, CNN-BiLSTM inference, trajectory planning, and robot control

Motor Specs

Parameter	Specification
Operating Voltage	4.8–7.2 V DC
Stall Torque (4.8 V)	9.4 kg·cm (0.92 N·m)
Stall Torque (6.0 V)	11.0 kg·cm (1.08 N·m)
No-Load Speed (4.8 V)	0.19 s/60°
No-Load Speed (6.0 V)	0.15 s/60°
Gear Type	Metal gears
Control Signal	PWM (50 Hz, pulse width approximately 1–2 ms)
Weight	55 g
Total Units Deployed	6

References

[1] A. Phinyomark and E. Scheme, “EMG pattern recognition in the era of big data and deep learning,” Sep. 01, 2018, *MDPI AG*. doi 10.3390/bdcc2030021.

[2] F. Pérez-Reynoso, N. Farrera, C. Capetillo, N. Méndez-Lozano, C. González-Gutiérrez, and E. López-Neri, “Pattern Recognition of EMG Signals by Machine Learning for the Control of a Manipulator Robot,” *Sensors*, vol. 22, no. 9, May 2022, doi 10.3390/s22093424.

[3] M. Barbero, R. Merletti, and A. Rainoldi, *Atlas of Muscle Innervation Zones*. Springer Milan, 2012. doi 10.1007/978-88-470-2463-2.

[4] D. Yadav and K. Veer, “Recent trends and challenges of surface electromyography in prosthetic applications,” Aug. 01, 2023, *Springer Verlag*. doi 10.1007/s13534-023-00281-z.

[5] Z. Zhang, C. He, and K. Yang, “A novel surface electromyographic signal-based hand gesture prediction using a recurrent neural network,” Jul. 02, 2020, *MDPI AG*. doi 10.3390/s20143994.

[6] M. V. Soosaimariyan, H. L. Allasi, K. Chitra, and J. B. Gnanadurai, “Enhanced EMG-Based Hand Gesture Recognition by Using Generalized Deep Infomax Networks,” *J. Sens.*, vol. 2025, no. 1, 2025, doi 10.1155/js/9496890.

[7] J. Shin, A. S. M. Miah, S. Konnai, I. Takahashi, and K. Hirooka, “Hand gesture recognition using sEMG signals with a multi-stream time-varying feature enhancement approach,” *Sci. Rep.*, vol. 14, no. 1, Dec. 2024, doi 10.1038/s41598-024-72996-7.

[8] R. Fu, B. Zhang, H. Liang, S. Wang, Y. Wang, and Z. Li, “Gesture recognition of sEMG signal based on GASF-LDA feature enhancement and adaptive ABC optimized SVM,” *Biomed. Signal Process. Control*, vol. 85, Aug. 2023, doi 10.1016/j.bspc.2023.105104.

[9] A. Phinyomark, R. N. Khushaba, and E. Scheme, “Feature Extraction and Selection for Myoelectric Control Based on Wearable EMG Sensors,” *Sensors (Basel)*, vol. 18, no. 5, May 2018, doi 10.3390/S18051615.

[10] A. Bahador, M. Yousefi, M. Marashi, and O. Bahador, “High accurate lightweight deep learning method for gesture recognition based on surface electromyography,” *Comput. Methods Programs Biomed.*, vol. 195, Oct. 2020, doi 10.1016/j.cmpb.2020.105643.

[11] L. I. Barona López, F. M. Ferri, J. Zea, Á. L. Valdivieso Caraguay, and M. E. Benalcázar, “CNN-LSTM and post-processing for EMG-based hand gesture recognition,” *Intelligent Systems with Applications*, vol. 22, Jun. 2024, doi 10.1016/j.iswa.2024.200352.

[12] L. I. Barona López, F. M. Ferri, J. Zea, Á. L. Valdivieso Caraguay, and M. E. Benalcázar, “CNN-LSTM and post-processing for EMG-based hand gesture recognition,” *Intelligent Systems with Applications*, vol. 22, Jun. 2024, doi 10.1016/j.iswa.2024.200352.

[13] R. Fu, B. Zhang, H. Liang, S. Wang, Y. Wang, and Z. Li, “Gesture recognition of sEMG signal based on GASF-LDA feature enhancement and adaptive ABC optimized SVM,” *Biomed. Signal Process. Control*, vol. 85, Aug. 2023, doi 10.1016/j.bspc.2023.105104.

[14] F. Pérez-Reynoso, N. Farrera, C. Capetillo, N. Méndez-Lozano, C. González-Gutiérrez, and E. López-Neri, “Pattern Recognition of EMG Signals by Machine Learning for the Control of a Manipulator Robot,” *Sensors*, vol. 22, no. 9, May 2022, doi 10.3390/s22093424.

[15] Y. Xia, D. Qiu, C. Zhang, and J. Liu, “sEMG-based gesture recognition using multi-stream adaptive CNNs with integrated residual modules,” *Front. Bioeng. Biotechnol.*, vol. 13, p. 1487020, 2025, doi 10.3389/FBIOE.2025.1487020.

[16] L. Wang, J. Fu, H. Chen, and B. Zheng, “Hand gesture recognition using smooth wavelet packet transformation and hybrid CNN based on surface EMG and accelerometer signal,” *Biomed. Signal Process. Control*, vol. 86, Sep. 2023, doi 10.1016/j.bspc.2023.105141.

[17] A. Hassan Abdelmoneam Mechatronics and N. Zayed, “Pattern Recognition of sEMG Signals by CNN-BiLSTM Algorithm For Bio-robotics Applications-term memory (BiLSTM), Human-Machine Interface (HMI), Human-Robot Interaction (HRI), and Convolutional neural network (CNN).”

[18] E. Rahimian, S. Zabihi, A. Asif, D. Farina, S. F. Atashzar, and A. Mohammadi, "TEMGNet Deep Transformer-based Decoding of Upperlimb sEMG for Hand Gestures Recognition," Sep. 2021, [Online]. Available http //arxiv.org/abs/2109.12379

[19] T. Kishore, S. S. Remabai, C. Retnaswamy, and E. S. Paul, "Enhanced EMG-based Gesture Recognition using Hybrid CNN-BiLSTM Architecture with Channel Attention," *Biomedical and Pharmacology Journal*, vol. 18, pp. 315–329, Mar. 2025, doi 10.13005/bpj/3090.

[20] I. Ben Abdallah, Y. Bouteraa, and A. Alotaibi, "Hybrid EMG–NMES control for real-time muscle fatigue reduction in bionic hands," *Scientific Reports 2025 15 1*, vol. 15, no. 1, pp. 22467-, Jul. 2025, doi 10.1038/s41598-025-05829-w.

[21] M. Tavakoli, C. Benussi, P. Alhais Lopes, L. B. Osorio, and A. T. de Almeida, "Robust hand gesture recognition with a double channel surface EMG wearable armband and SVM classifier," *Biomed. Signal Process. Control*, vol. 46, pp. 121–130, Sep. 2018, doi 10.1016/j.bspc.2018.07.010.

[22] P. K. Artemiadis and K. J. Kyriakopoulos, "An EMG-based robot control scheme robust to time-varying EMG signal features," *IEEE Transactions on Information Technology in Biomedicine*, vol. 14, no. 3, pp. 582–588, May 2010, doi 10.1109/TITB.2010.2040832.

[23] S. Jain, A. Farshchiansadegh, A. Broad, F. Abdollahi, F. Mussa-Ivaldi, and B. Argall, "Assistive Robotic Manipulation through Shared Autonomy and a Body-Machine Interface," *IEEE Int. Conf. Rehabil. Robot.*, vol. 2015, pp. 526–531, Sep. 2015, doi 10.1109/ICORR.2015.7281253.

[24] E. J. Fabregó, "EMG-Based Control of a Robotic Arm," 2025.

[25] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2 Design, architecture, and uses in the wild," *Sci. Robot.*, vol. 7, no. 66, May 2022, doi 10.1126/SCIROBOTICS.ABM6074;ISSUE ISSUE DOI.

[26] U. Côté-Allard *et al.*, "Deep Learning for Electromyographic Hand Gesture Signal Classification Using Transfer Learning," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 27, no. 4, pp. 760–771, Apr. 2019, doi 10.1109/TNSRE.2019.2896269.

[27] T. J. M. Dick *et al.*, “Consensus for experimental design in electromyography (CEDE) project Application of EMG to estimate muscle force,” *Journal of Electromyography and Kinesiology*, vol. 79, Dec. 2024, doi 10.1016/j.jelekin.2024.102910.